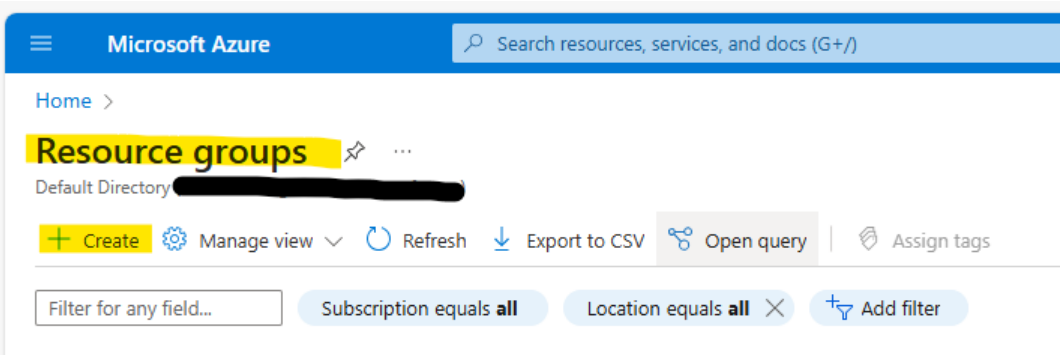
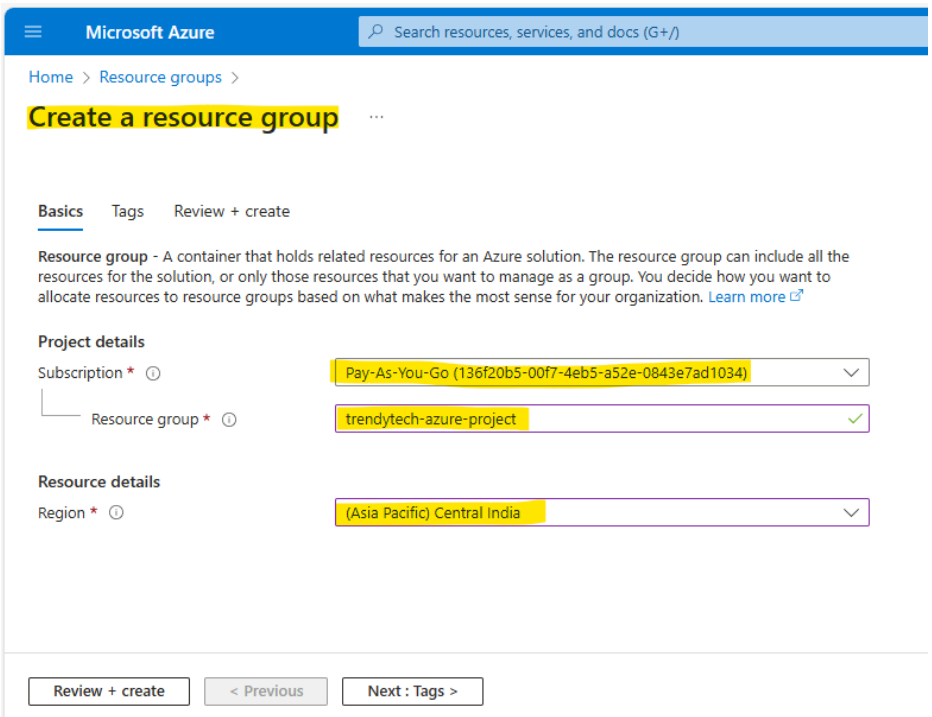
Part 1:

Before creating all the resources we will create the resource group in which we will create all the required resources.

Go to Azure Portal and log in with your Azure account. In the left-hand menu, select "Resource groups". If you don't see it, use the search bar at the top of the page and search for "Resource groups". Click the "+ Create" button or "Add" at the top of the Resource groups page.



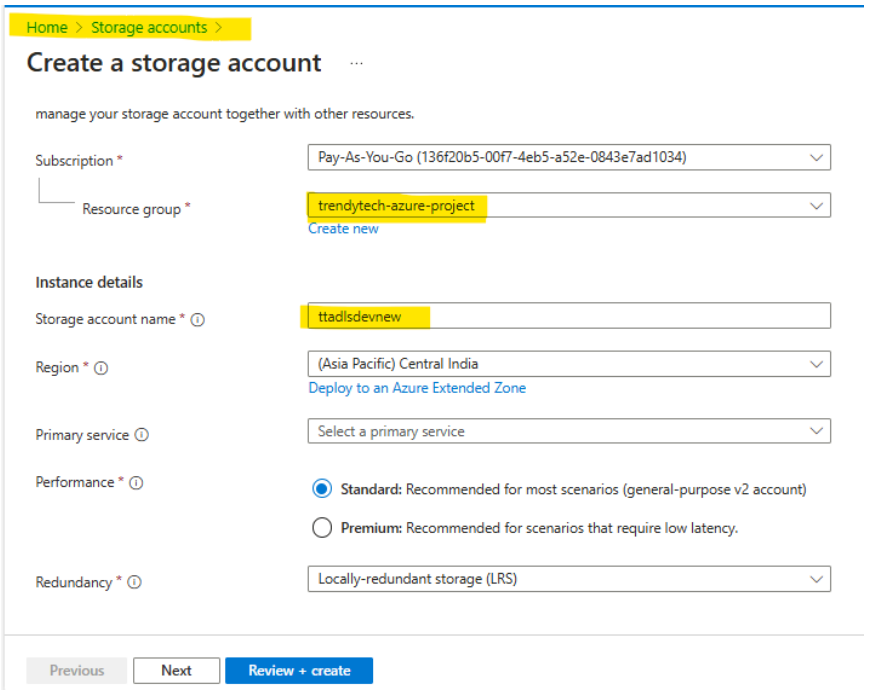
Select the subscription under which the resource group will be created. Enter a unique name for your resource group. Choose a location (region) where your resources will reside (e.g., East US, West Europe).



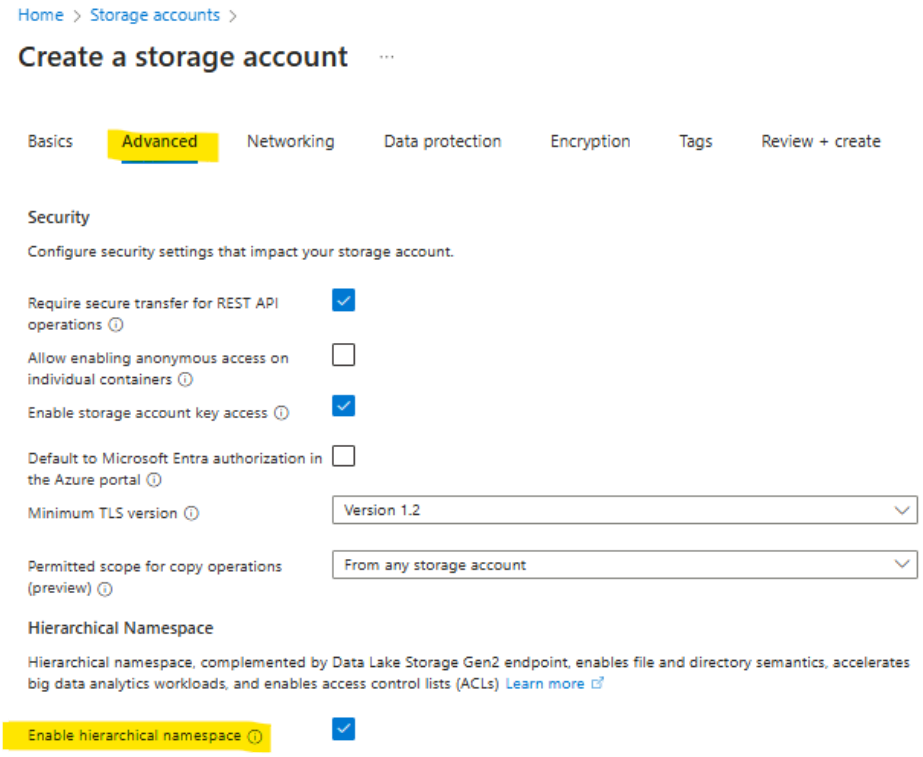
Click "Review + Create" and then "Create".

Azure Storage Account creation:

Search for "Storage accounts" and click "+ Create". Select subscription, resource group, region, and enter a unique storage account name as shown below.

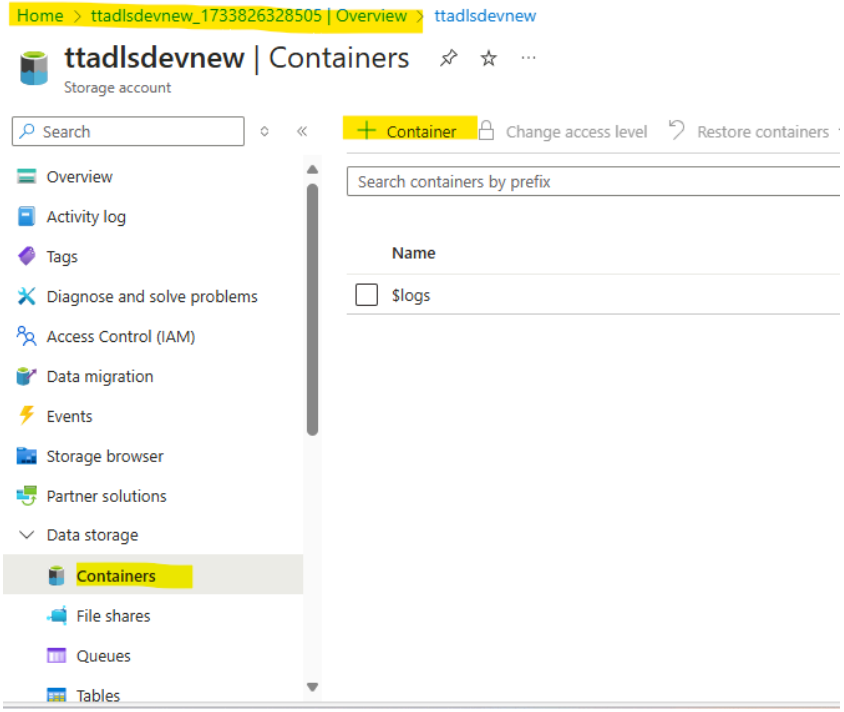


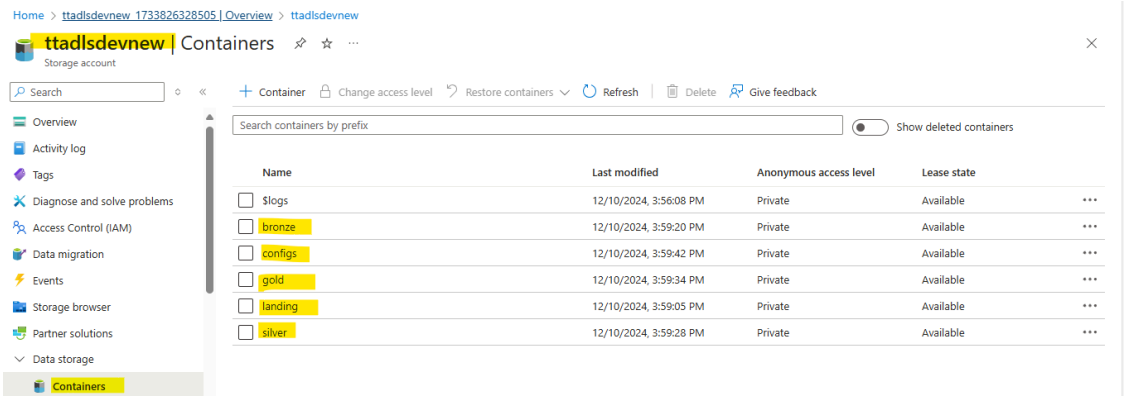
Enable ADLS Gen2: Go to the Advanced tab and enable Hierarchical namespace.



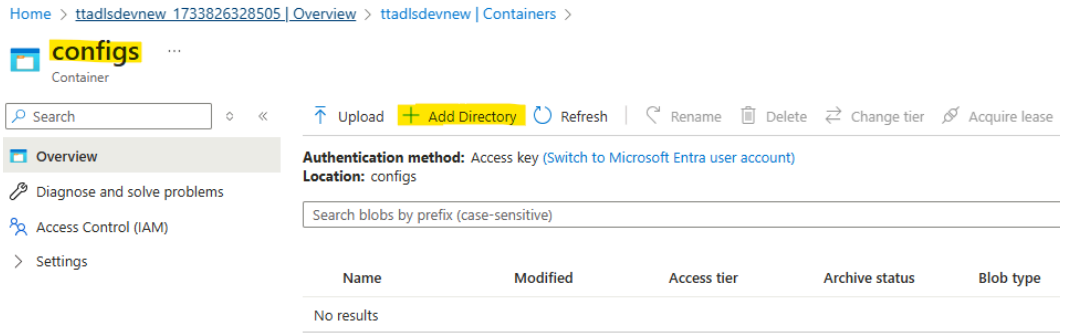
Click Review + create, validate settings, and click Create.

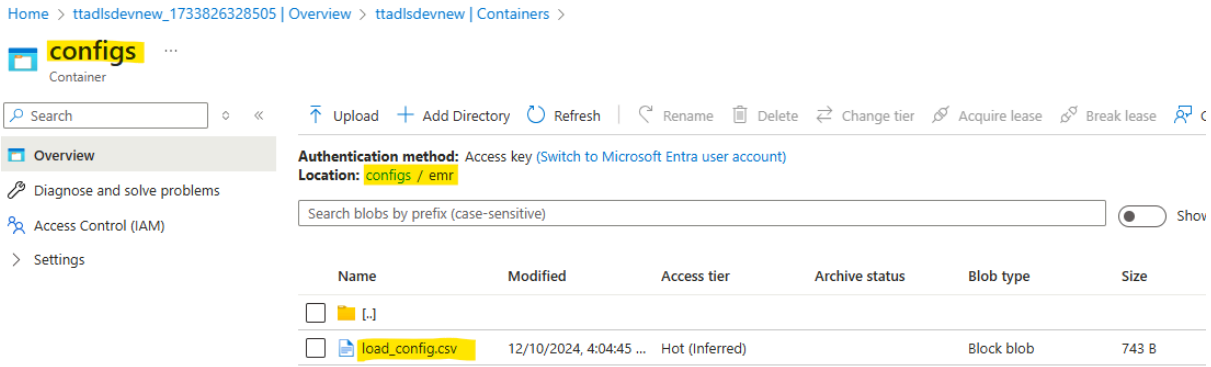
Now create the containers “landing”, “bronze”, “silver”, “gold”, “configs” in this storage account as shown below.





Then in the configs container create the directory “emr” and then upload the file “load_config.csv” in it.

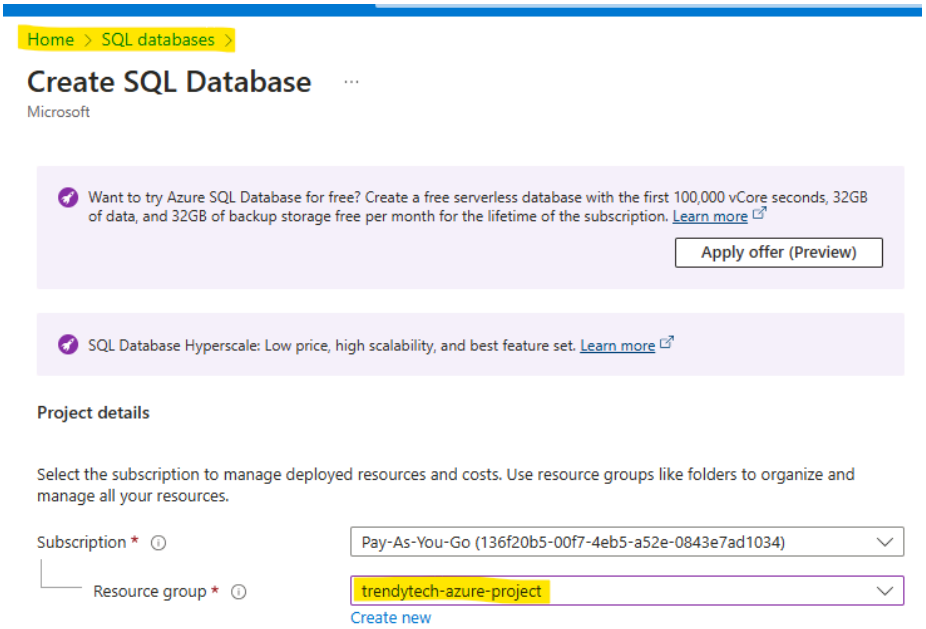




Steps to create Azure SQL database:

We will create 2 azure SQL db - trendytech-hospital-a, trendytech-hospital-b

In the search bar, type "SQL Database" and select "SQL Database" from the results. Choose your Subscription and Resource Group. Enter a Database Name. Also create a SQL Server.



Database details

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources

Database name * trendytech-hospital-a

Server * Select a server

Create new

Compute + storage * Please select a server first.

Configure database

We will create the server as shown below. Choose a Compute + Storage tier (e.g., Basic, General Purpose).

Home > SQL databases > Create SQL Database >

Create SQL Database Server

Microsoft

Server details

Enter required settings for this server, including providing a name and location. This server will be created in the same subscription and resource group as your database.

Server name * trendytech-sqlserver

.database.windows.net

Location * (Asia Pacific) Central India

Authentication

Azure Active Directory (Azure AD) is now Microsoft Entra ID. Learn more

Select your preferred authentication methods for accessing this server. Create a server admin login and password to access your server with SQL authentication, select only Microsoft Entra authentication Learn more using an existing Microsoft Entra user, group, or application as Microsoft Entra admin Learn more, or select both SQL and Microsoft Entra authentication.

Authentication method

- ○ Use Microsoft Entra-only authentication
- ○ Use both SQL and Microsoft Entra authentication
- ◉ Use SQL authentication

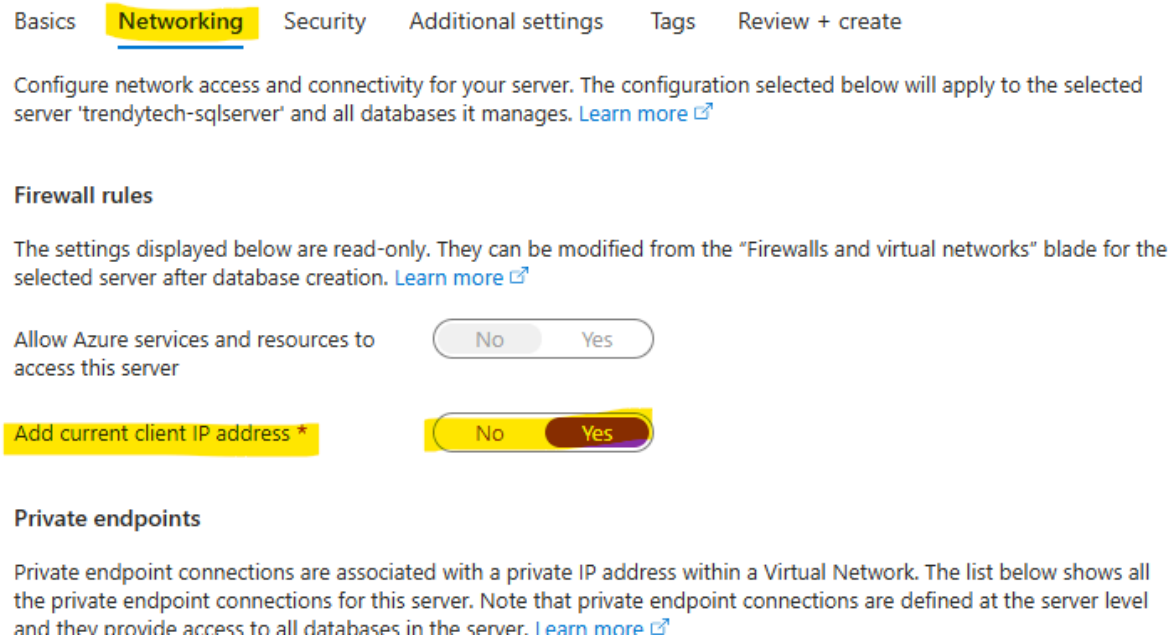
Server admin login * trendytechadmin

Password * ••••••••

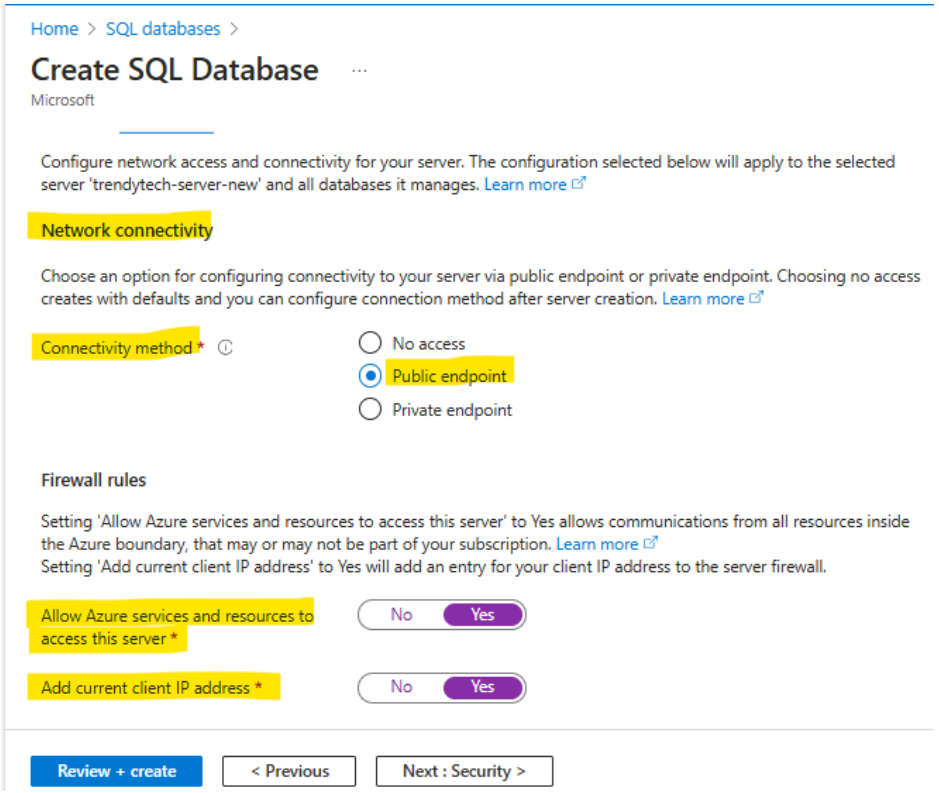
Confirm password * ••••••••

OK

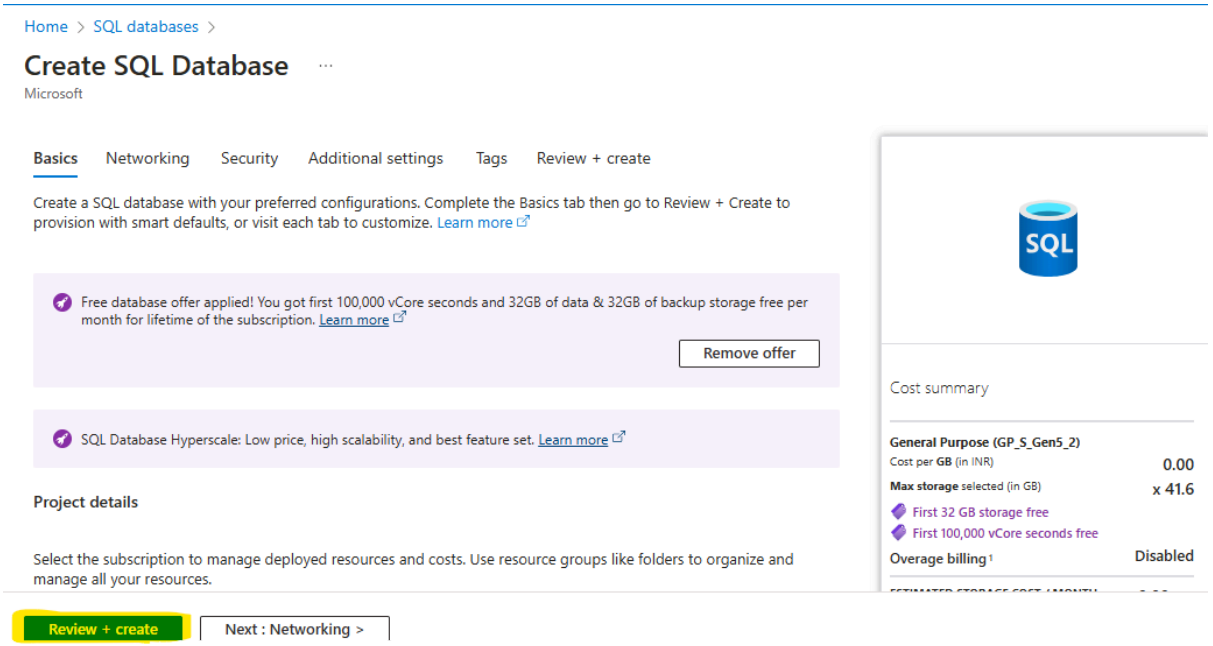
After this Go to Networking => In Network connectivity select “Public endpoint” option. Also set yes for the options “Allow Azure services and resources to access this server” and “Add current client IP address”



Note: Please note down this username and password for future reference.

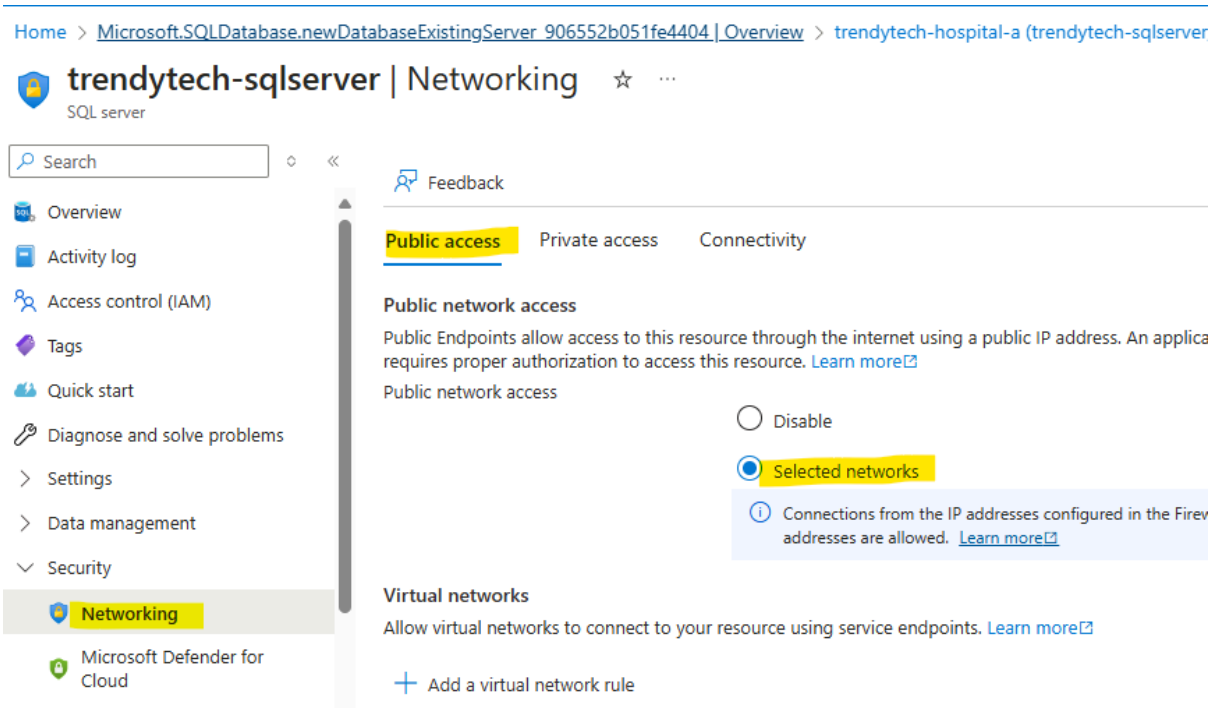


Click Review + Create, validate the details, and then click Create as shown below.

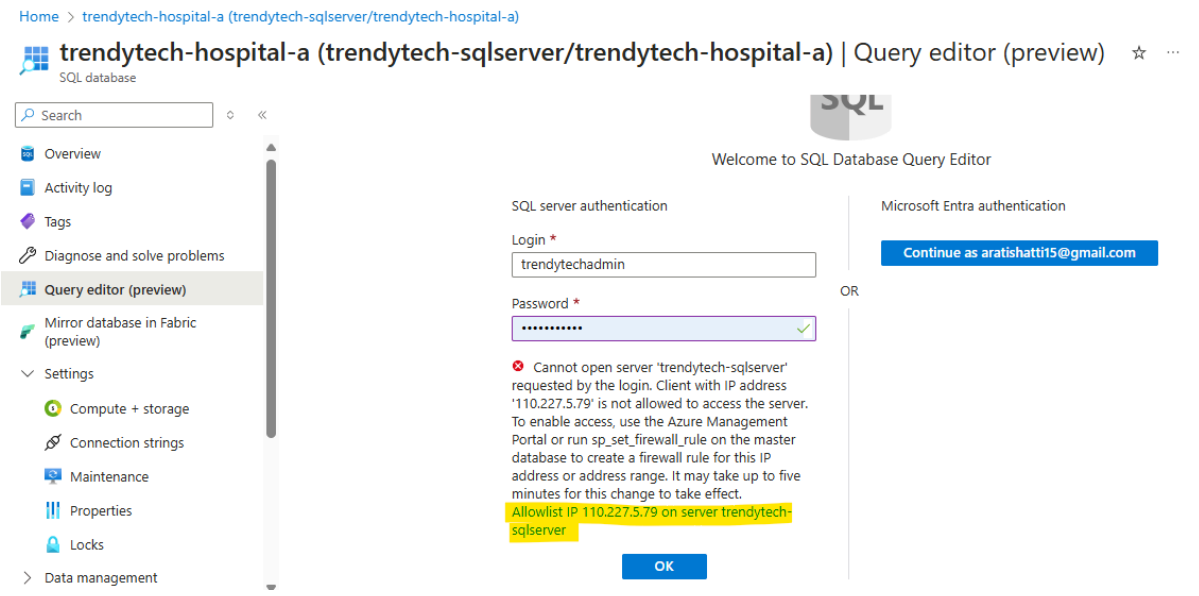


Note: While creating database if you are not able to allow public access and add client ip address you can follow below steps:

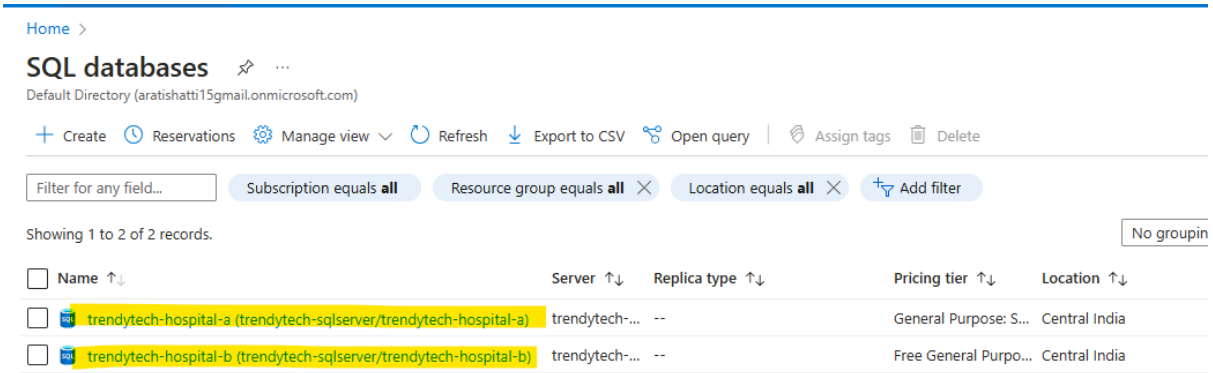
After creating this database Go to Networking => For Public access (select option Selected networks and save this) =>



Also while using query editor if you face below error click on “Allowing IP for current ip address” as shown below



Similarly we will create another database trendytech-hospital-b(We will use the same server i.e trendytech-sqlserver that we have created while creating trendytech-hospital-a database). Thus we have created 2 databases as shown below.

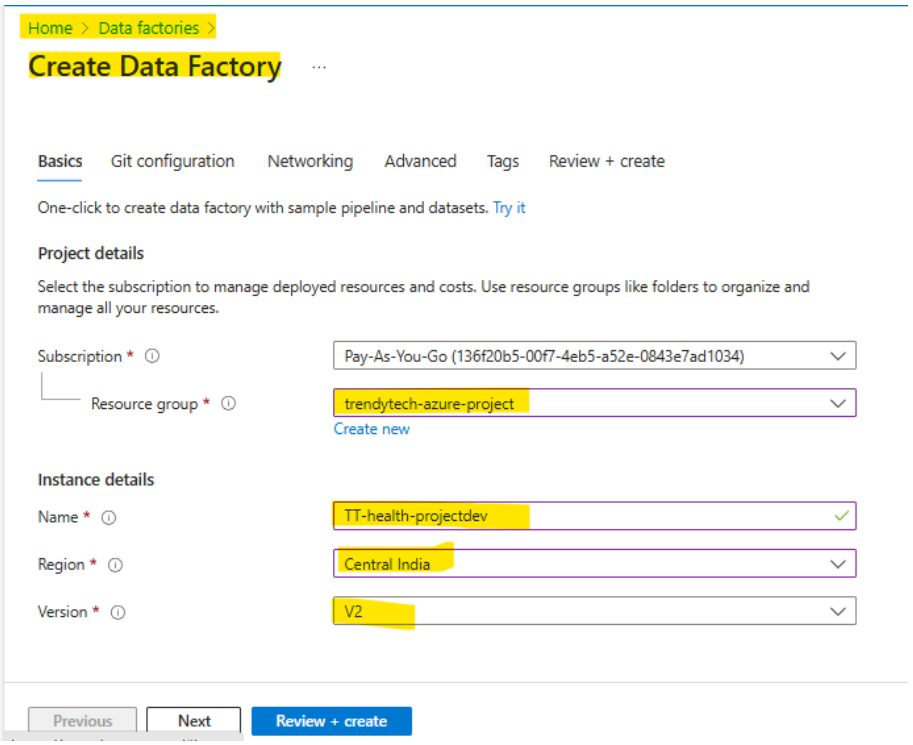


Then we will create the tables in these databases and for creating tables in the database use below scripts which are present on github account:
For trendytech-hospital-a =>
Trendytech_hospital_A_table_creation_commands

For trendytech-hospital-b =>
Trendytech_hospital_B_table_creation_commands

Steps to create ADF:

In the search bar, type "Data Factory" and select "Data Factory" from the results. Click the "Create" button on the Data Factory page. Provide a globally unique name for your Data Factory instance. Choose V2 (Data Factory Version 2) for the latest features.



Click "Review + Create" to validate the details. If validation passes, click "Create" to deploy the Data Factory.

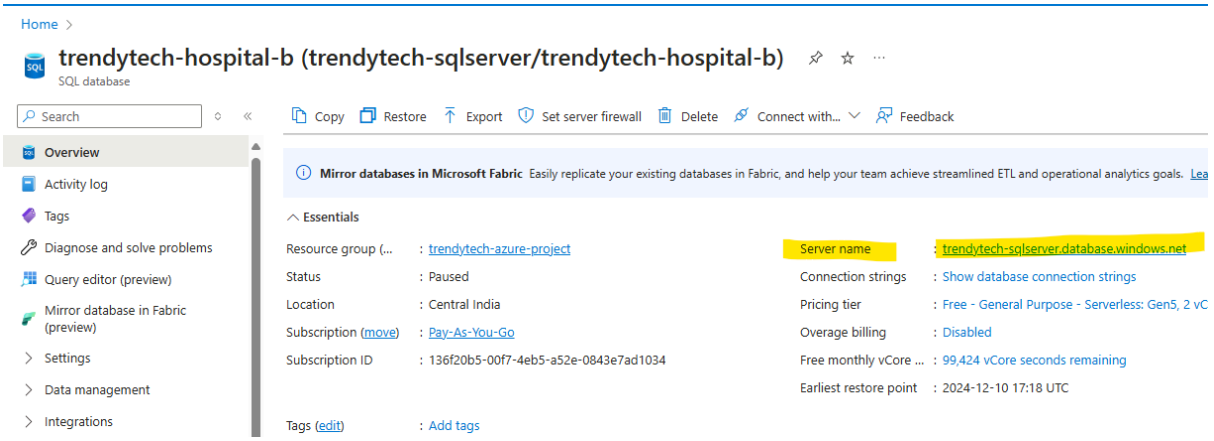
Steps to create ADF pipeline:

Linked Services creation:

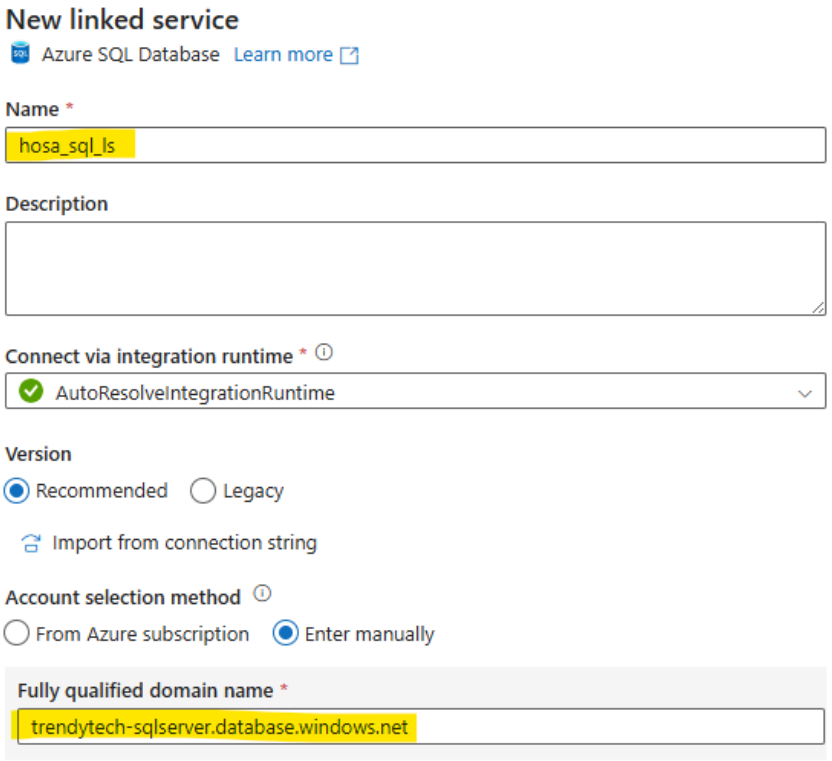
In the ADF interface, go to the Manage section on the left-hand panel. Under the Connections section, select Linked Services. Click on New to create a new Linked Service.

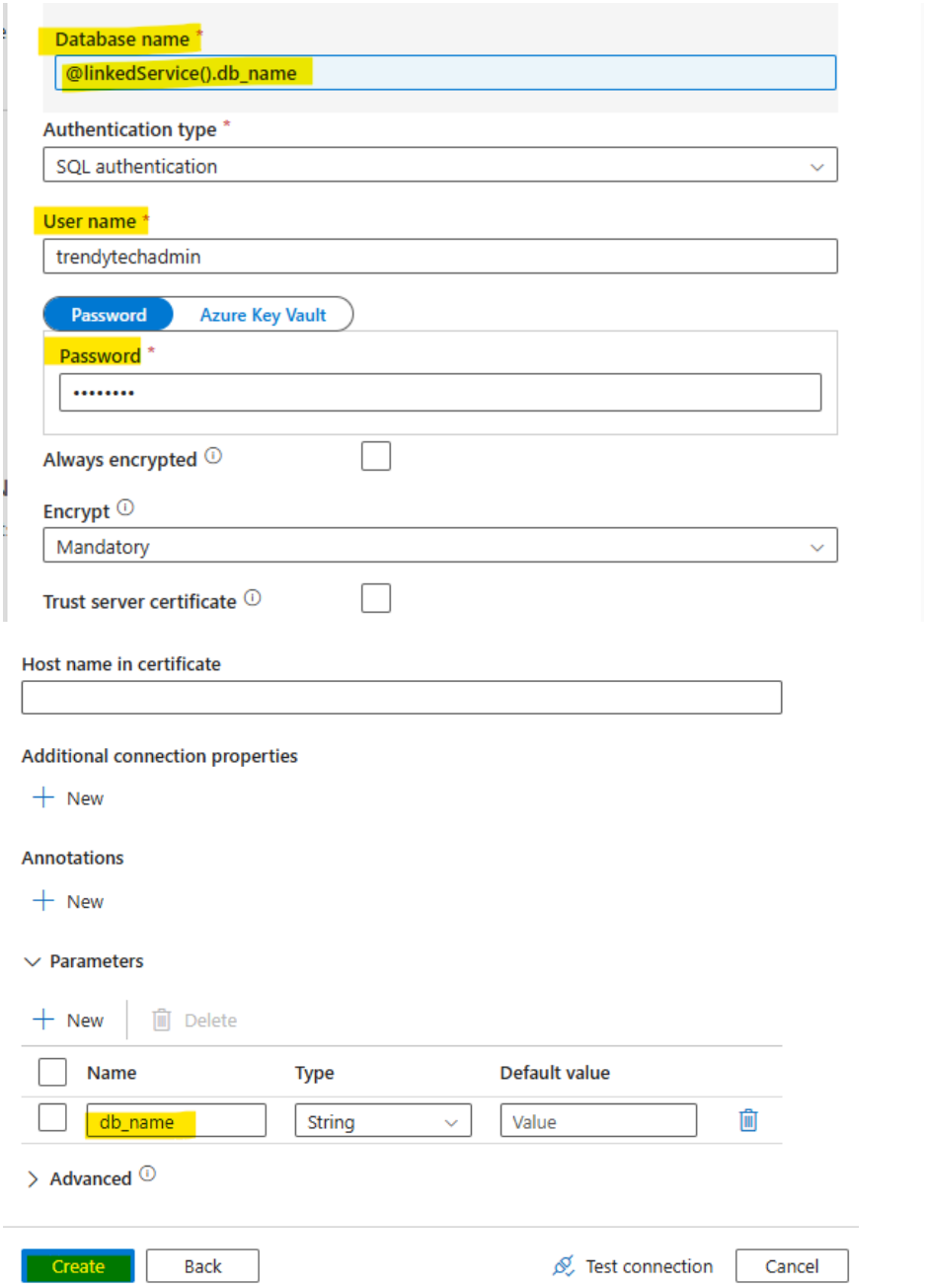
1. Azure SQL DB

Note down the server name for that sql database that we have created.



In fully qualified domain names, mention the server name, mention the username and password for sql server and define the parameter db_name and using this parameter we will pass the database name as shown below.



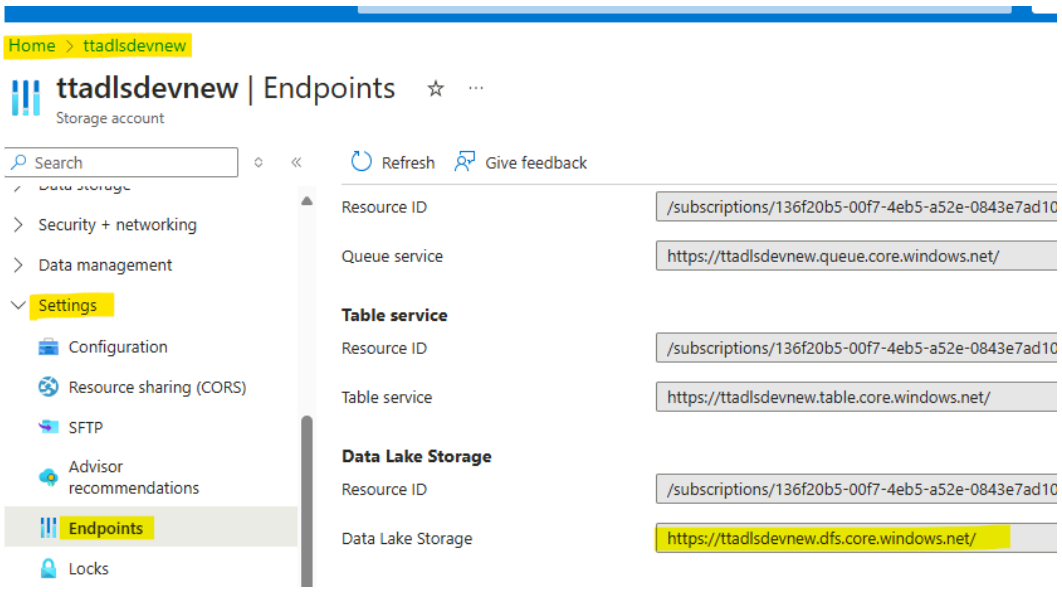


And click on create to create the linked service.

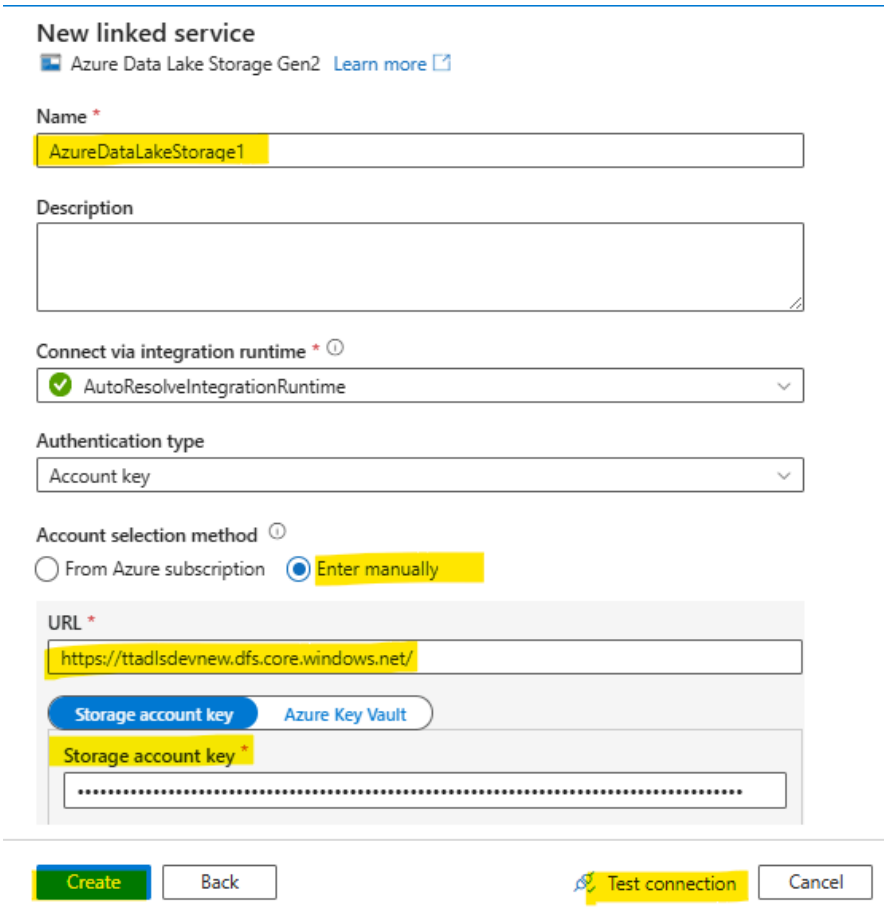
2. ADLS GEN2

Select Azure Data Lake Storage1 as the data store. Provide the following details- Name of your Blob Storage account, Authentication. Then click Test Connection to verify and save the Linked Service.

To get the url for Azure Data Lake Storage go to Adls gen2 storage that we have created => Setting => Endpoints => and copy the URL as shown below



Also copy the access key. Using these details create the linked service as shown below.

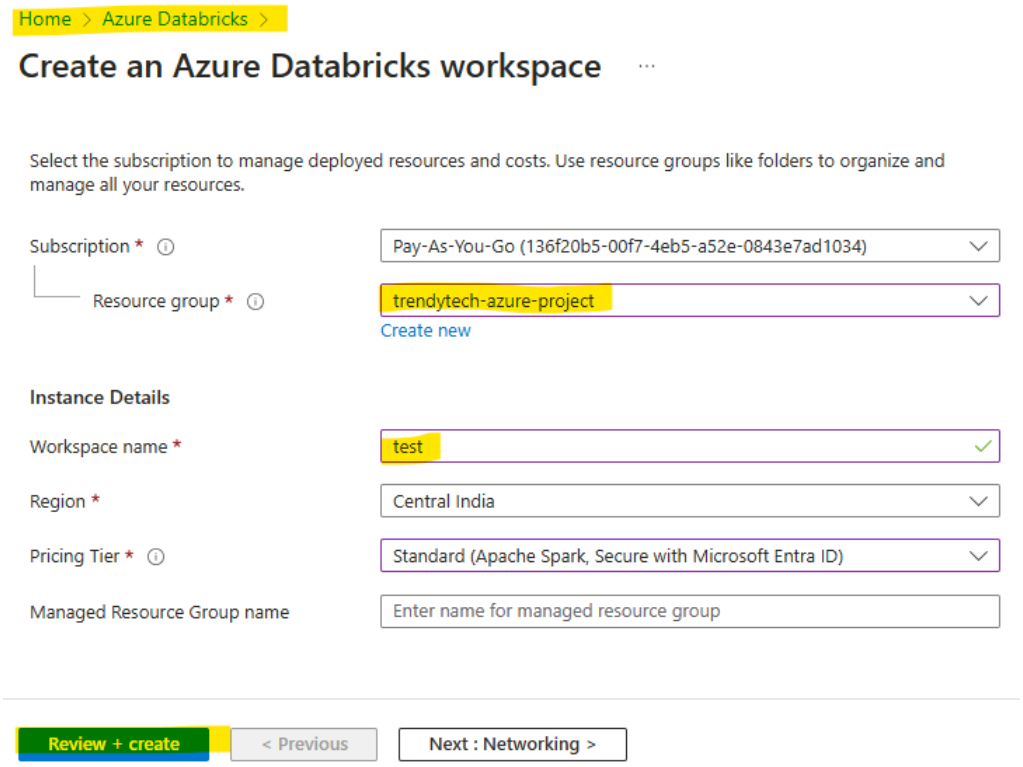


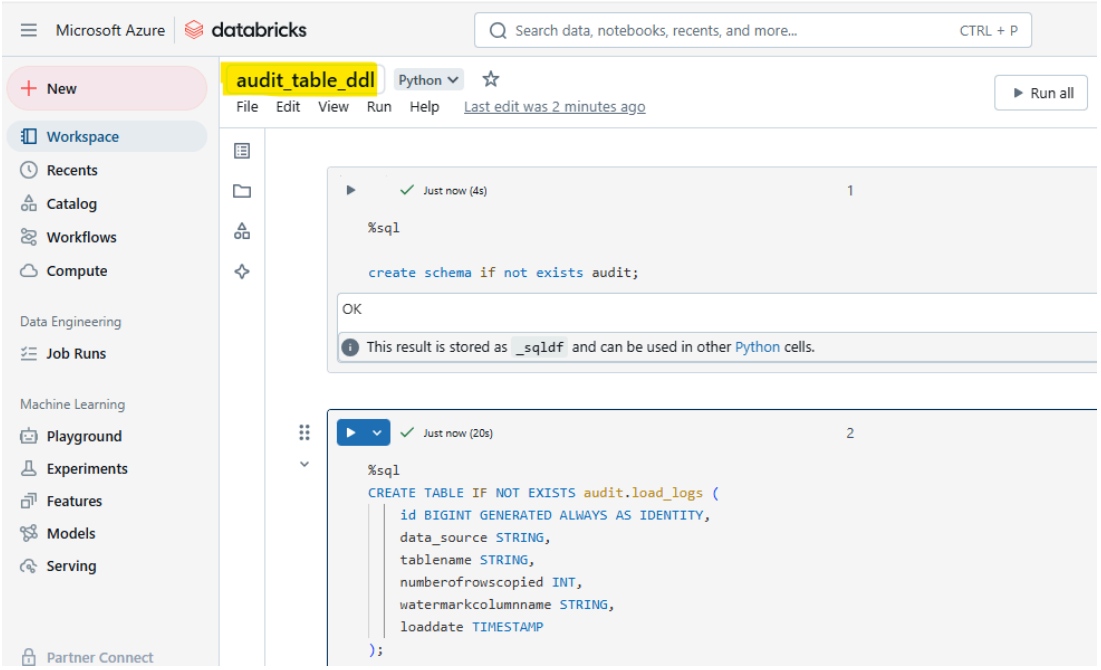
3. Delta table - Audit_logs

Create the databricks workspace test and then upload the code notebook “Audit_table_DDL” and start your databricks cluster and create the schema audit table in it using below commands. (Notebook name - audit_table_ddl)

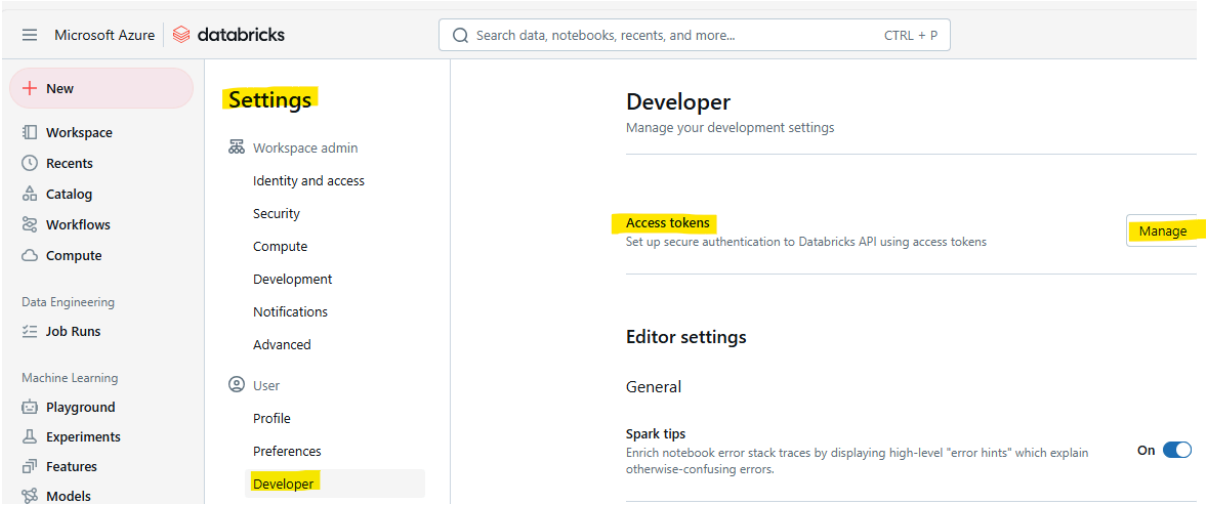
```
create schema if not exists audit;

CREATE TABLE IF NOT EXISTS audit.load_logs (
   id BIGINT GENERATED ALWAYS AS IDENTITY,
   data_source STRING,
   tablename STRING,
   numberofrowscopied INT,
   watermarkcolumnname STRING,
   loaddate TIMESTAMP
);
```



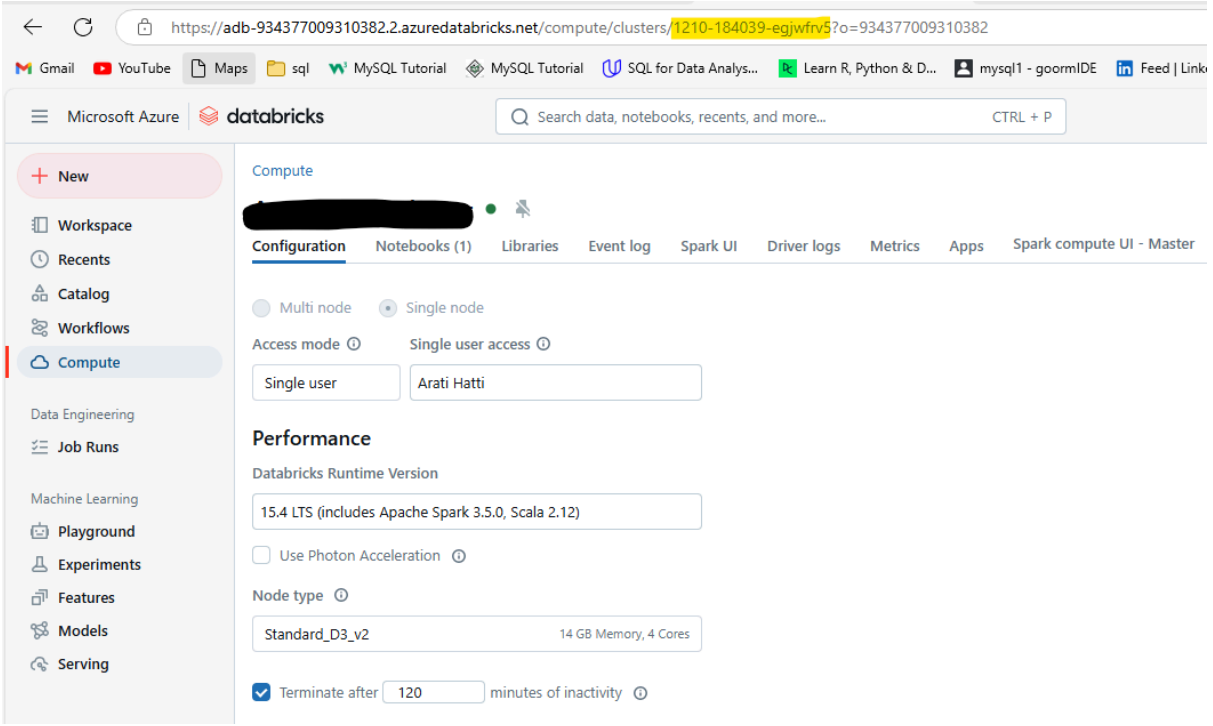


Note: To get access token Click your profile icon (top-right corner of the workspace) => Select Settings => Developer (In user setting) => Generate Access Token as shown below

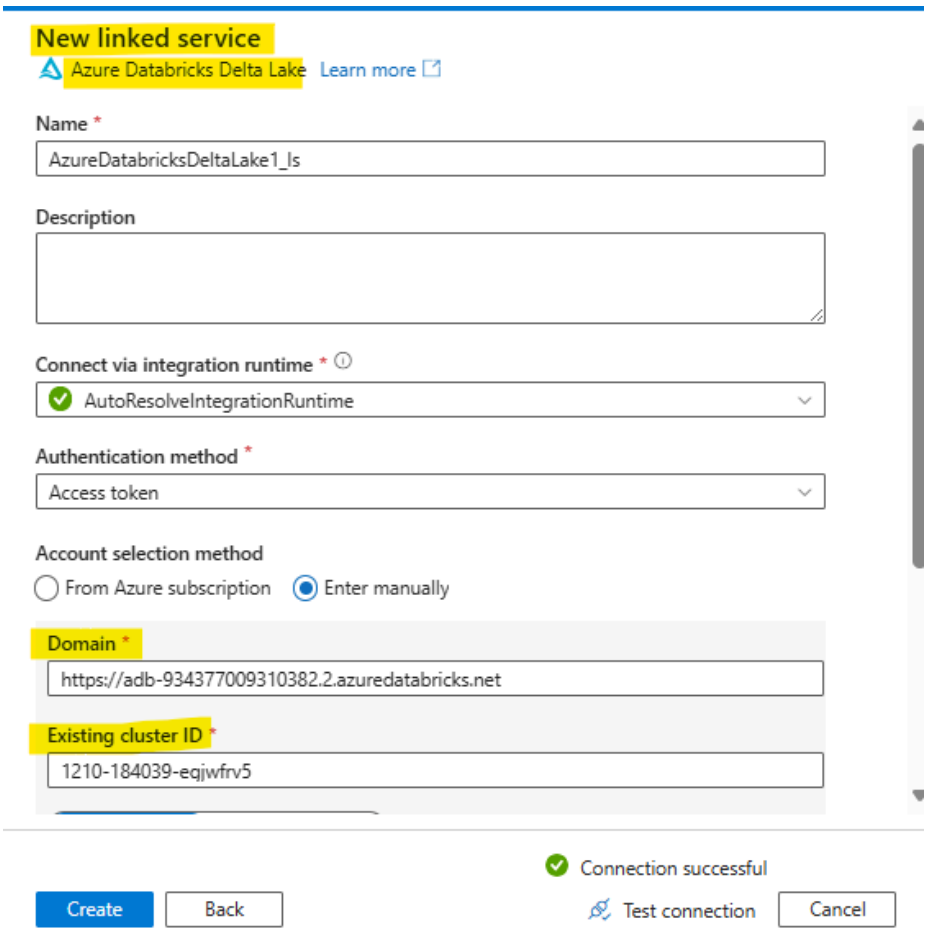


Then generate the token and copy for Future use.

While creating linked service in source mention “AzureDatabricksDeltaLake”. Then in the domain mention the URL of databricks workspace. And mention the cluster id for the cluster that we have created. To get Datbricks overview page to get Workspace URL and to get cluster id go to compute => Select the cluster => copy the cluster id as shown below



Refer below screenshot for more details.

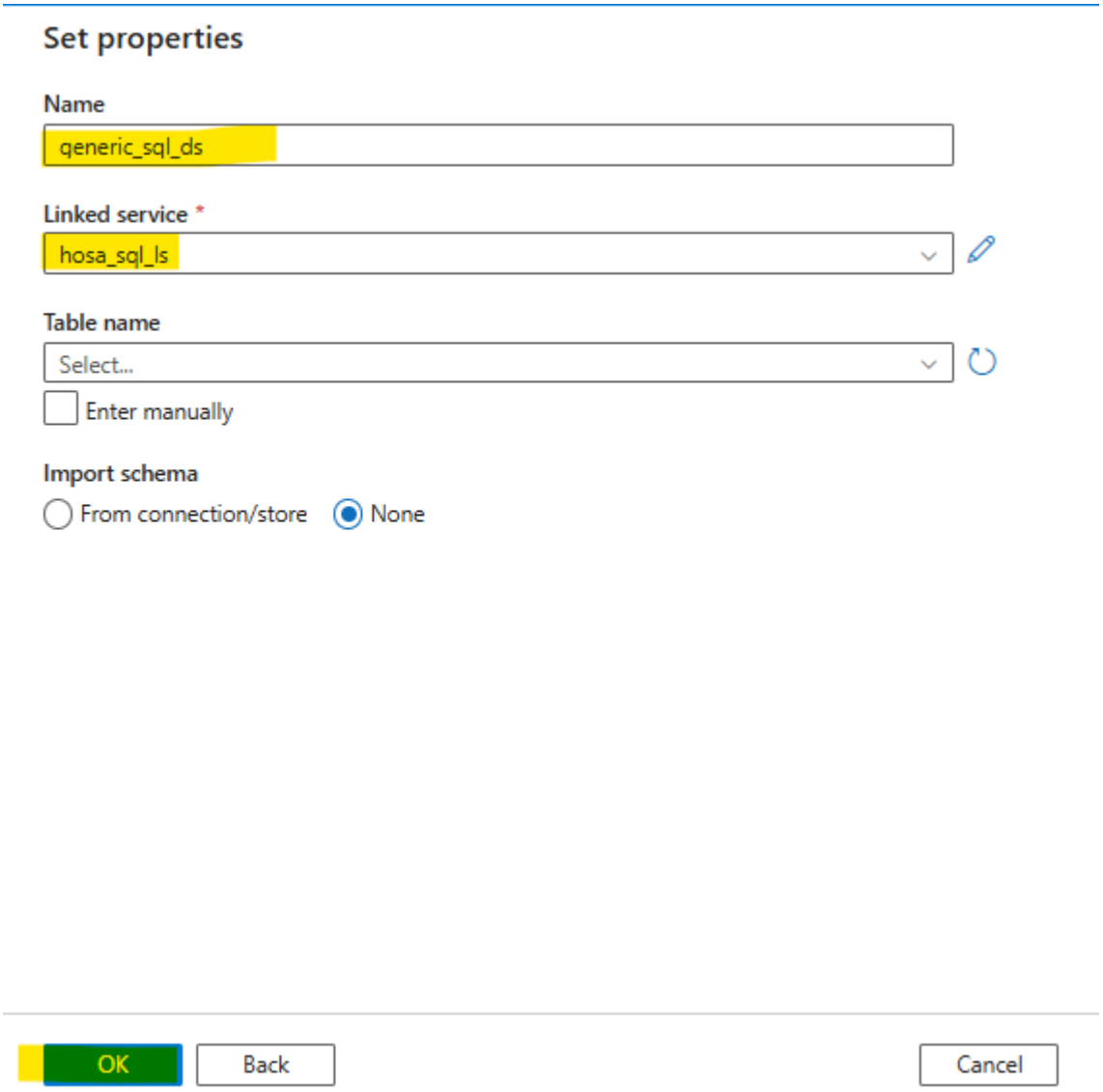


Dataset creation:

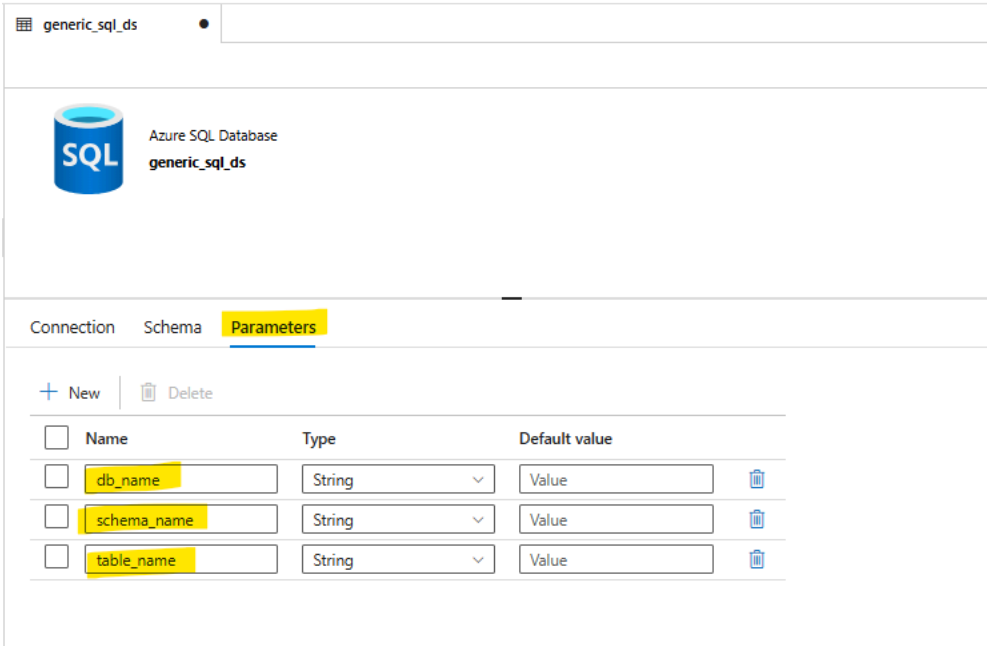
In the ADF interface, click on the Author section (left-hand panel). Expand the Datasets option. Click on the "…" next to Datasets in order to create the dataset.

1. Azure SQL DB

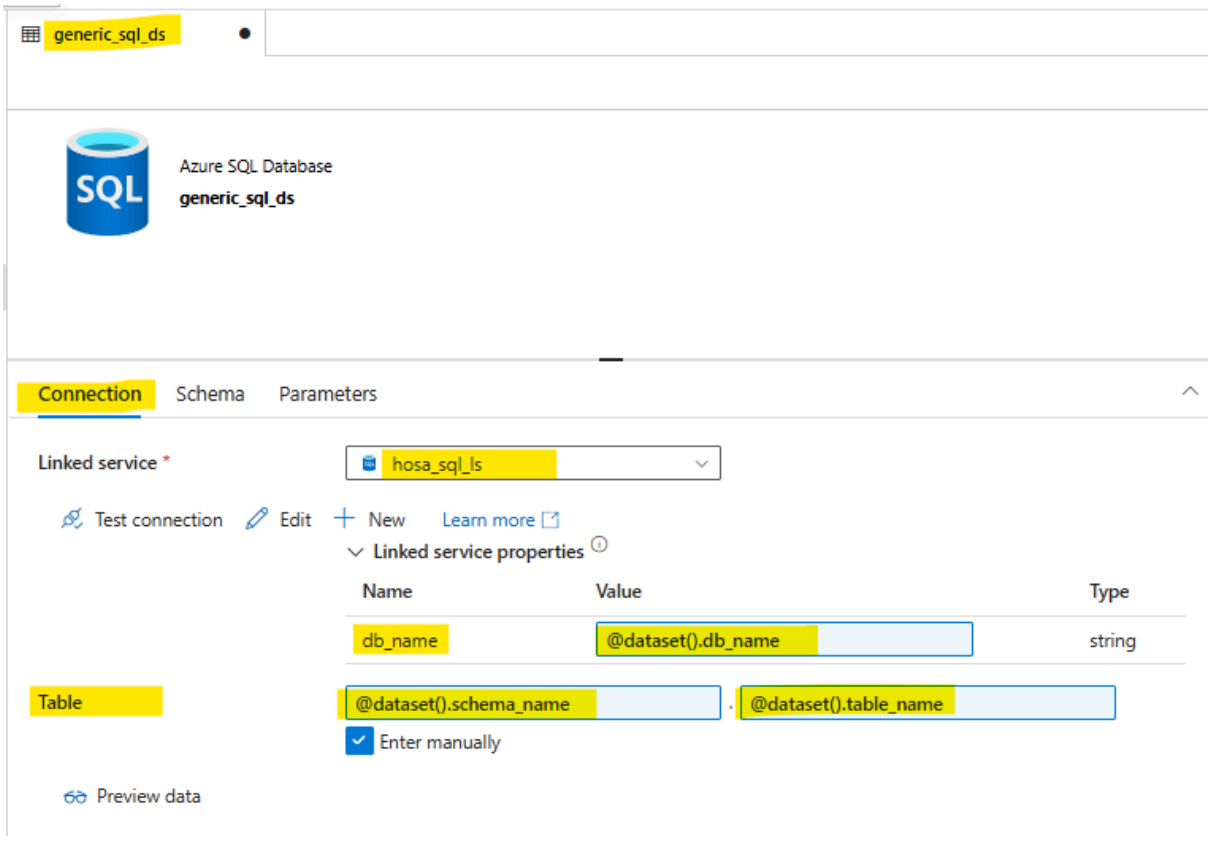
We will select the linked service that we have created for the SQL database. To create the datasets for the tables in a parameterized way in the sql database , we will create the parameter db_name, schema_name, table_name.



Then we will create parameters db_name, schema_name, table_name as shown below.

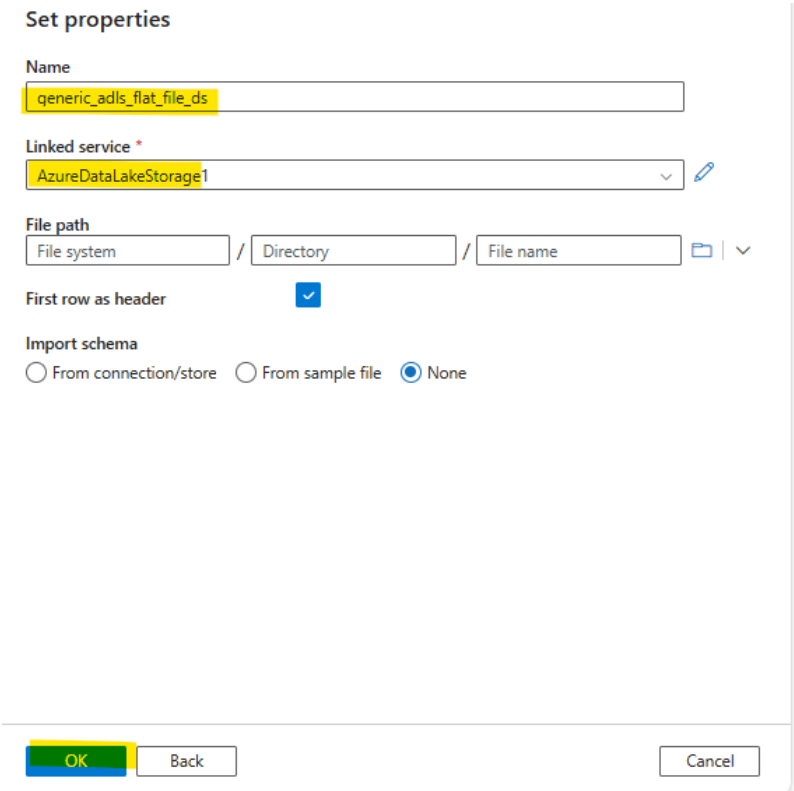


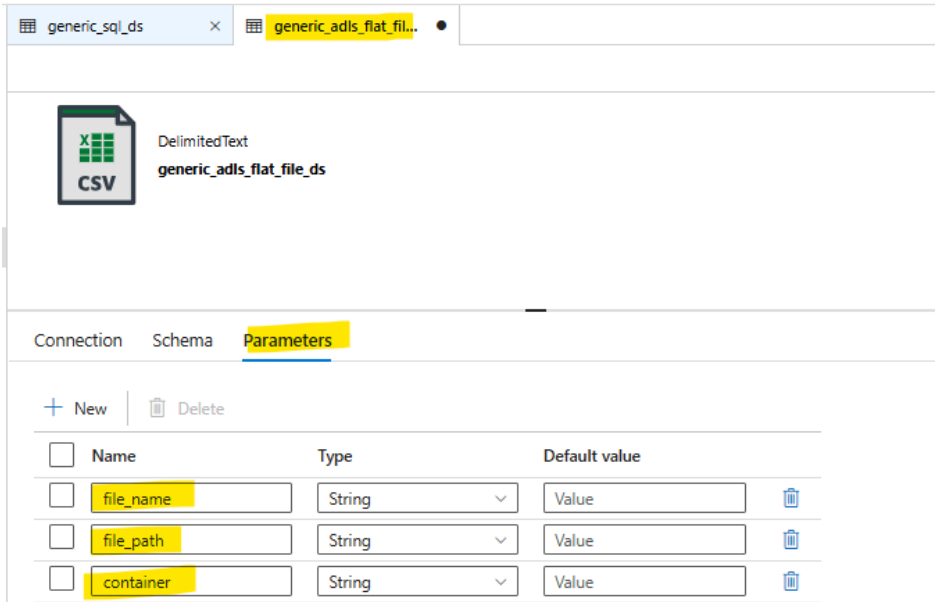
And we will pass dynamic value for table name and schema name as shown below

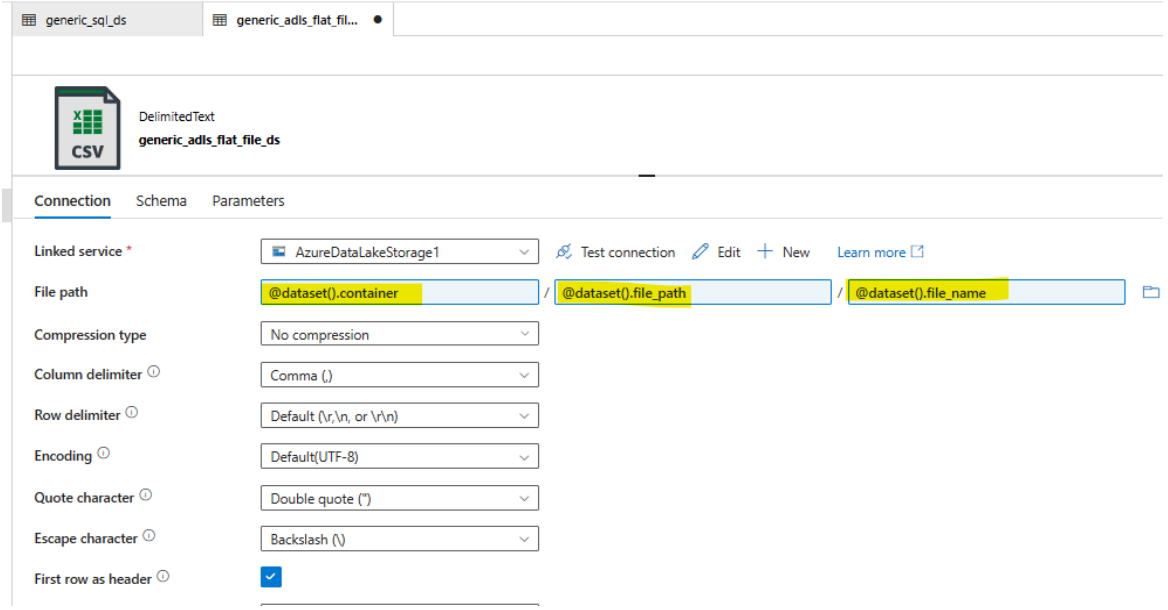


2. Dataset for Flatfile in ADLS GEN2

Select source as ADLS gen2 and file format as delimited text. Also in order to make it generic we will create the parameter file_name, file_path and container.



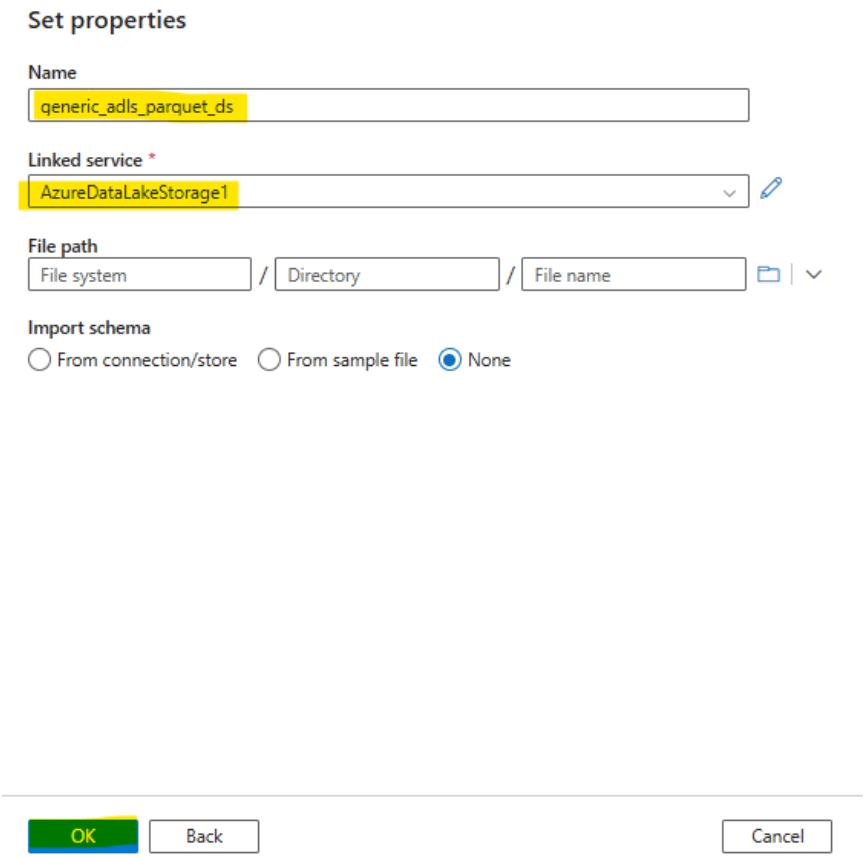


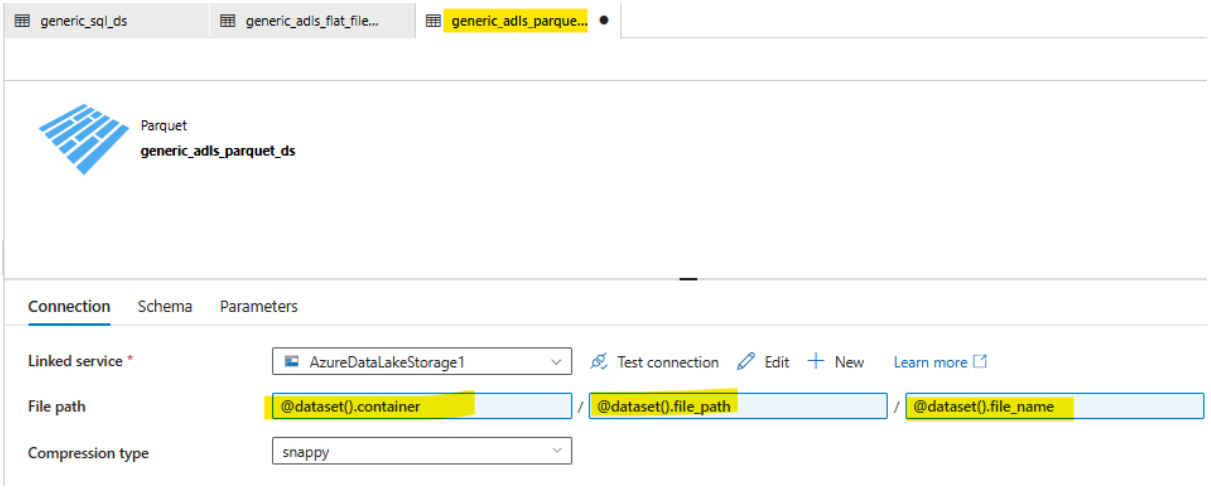


Now publish the changes.

3. Dataset for Parquet file in ADLS GEN2

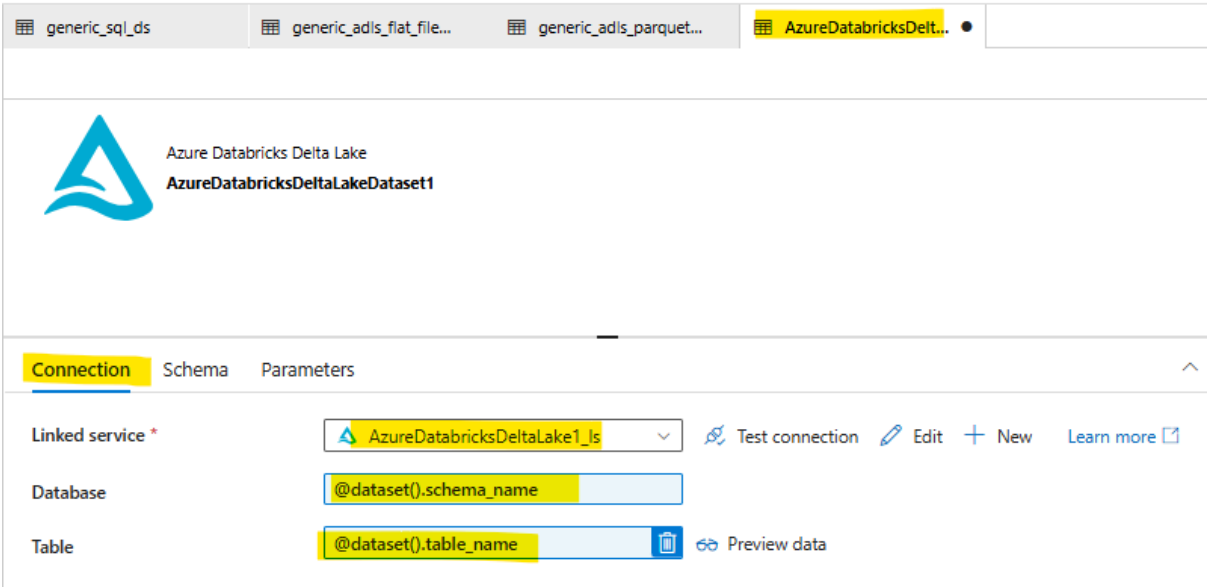
In order to store data in ADLS gen2 in parquet format we will need the dataset.
While creating this dataset we will select source as ADLS gen2, fileformat as parquet and we will create parameters file_name, file_path and container.





4. Databricks Delta Lake for Delta lake

We will select the source as Azure Databricks Delta Lake. For this we will create the parameter schema_name and table_name.



Once all the dataset and linked service are created, publish all in order to save them.

Creation of Pipelines:

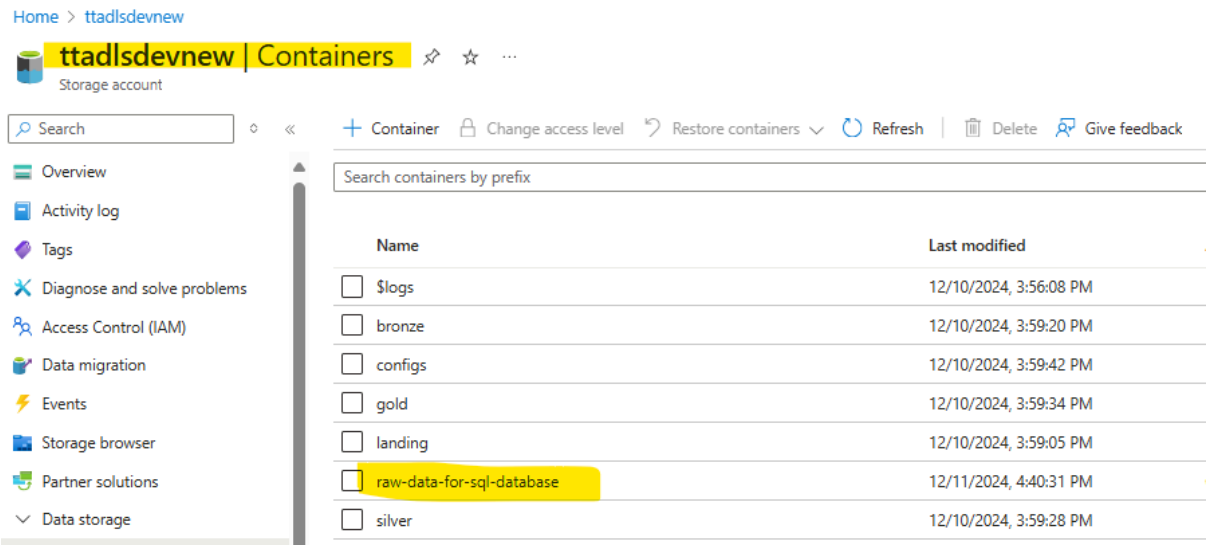
Background activity : Creation of pipeline to copy data into sql tables (pl_to_insert_data_to_sql_table_preprocessing).

Before proceeding with the main pipeline, we will create a simple pipeline in Azure Data Factory (ADF) to copy data from ADLS Gen2 storage into tables in an SQL database. This serves as a prerequisite to ensure that the SQL tables contain the data needed for the main pipeline.

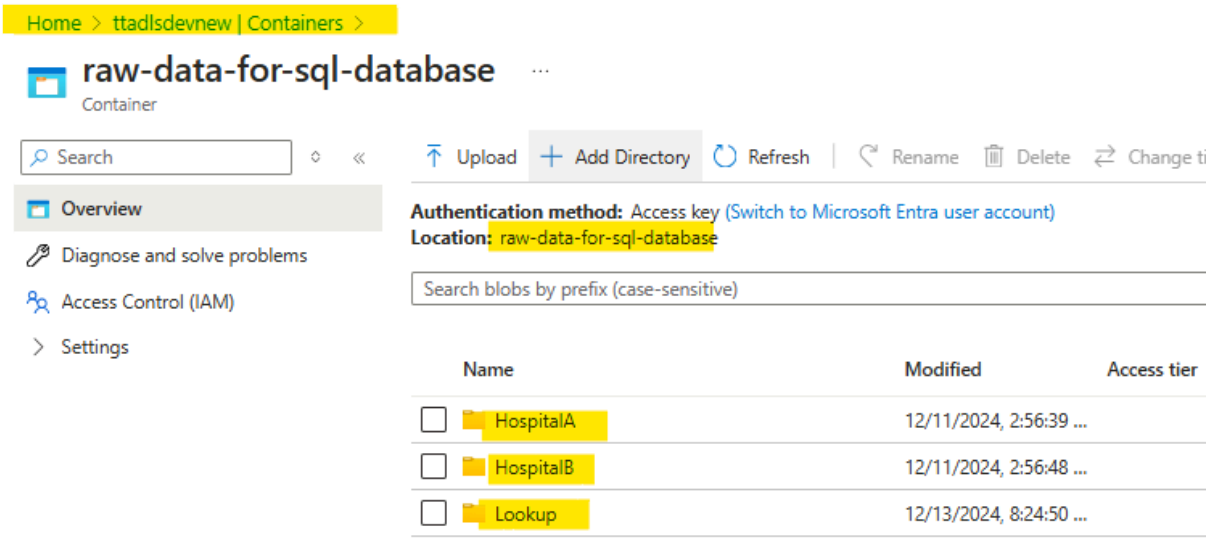
Note: We will create a new container (raw-data-for-sql-database) in the given ADLS Gen2 storage (adlsdevnew) and upload our CSV files, which will serve as the source for the pipeline, along with a lookup file. Additionally, we will create a dataset for the lookup file to use in this pipeline. Using the copy activity in ADF, we will transfer the data into the following tables: Departments, Providers, Encounters, Patients, and Transactions, located in the SQL databases trendytech-hospital-a and trendytech-hospital-b.

Source: ADLS gen2 -adlsdevnew

We will create a new container (raw-data-for-sql-database) in the given ADLS Gen2 storage (adlsdevnew) and upload our CSV files, along with a lookup file.

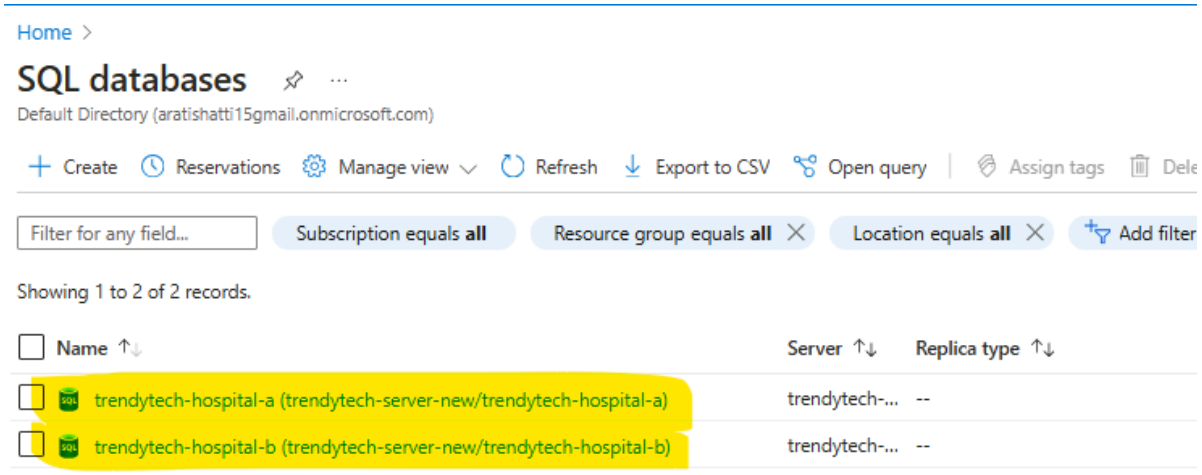


Folder: HospitalA, HospitalB for datafiles, Lookup for lookup file



Sink: SQL DB - trendytech-hospital-a, trendytech-hospital-b:

Note: We have already created these databases so no need to create again.



Pipeline creation Steps:

1. **Creation of Linked Services:**
   => For ADLS gen2 storage(source):

We will use the same linked service “AzureDataLakeStorage1” that we have created earlier for ADLS Gen2 storage.

=> For SQL DB(Sink):

We will use the same linked service “hosa_sql_ls” that we have created earlier for the database.

2. Creation of Datasets:

=> For source:

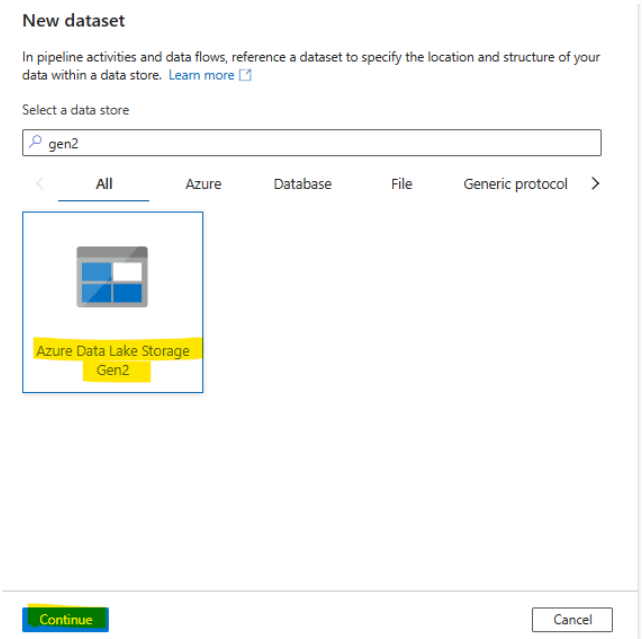
We will use the same generic dataset “generic_adls_flat_file_ds” that we have created earlier.

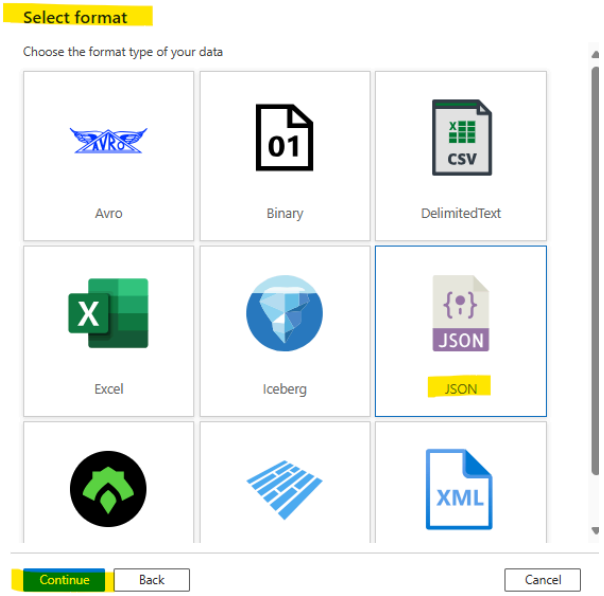
=> For sink:

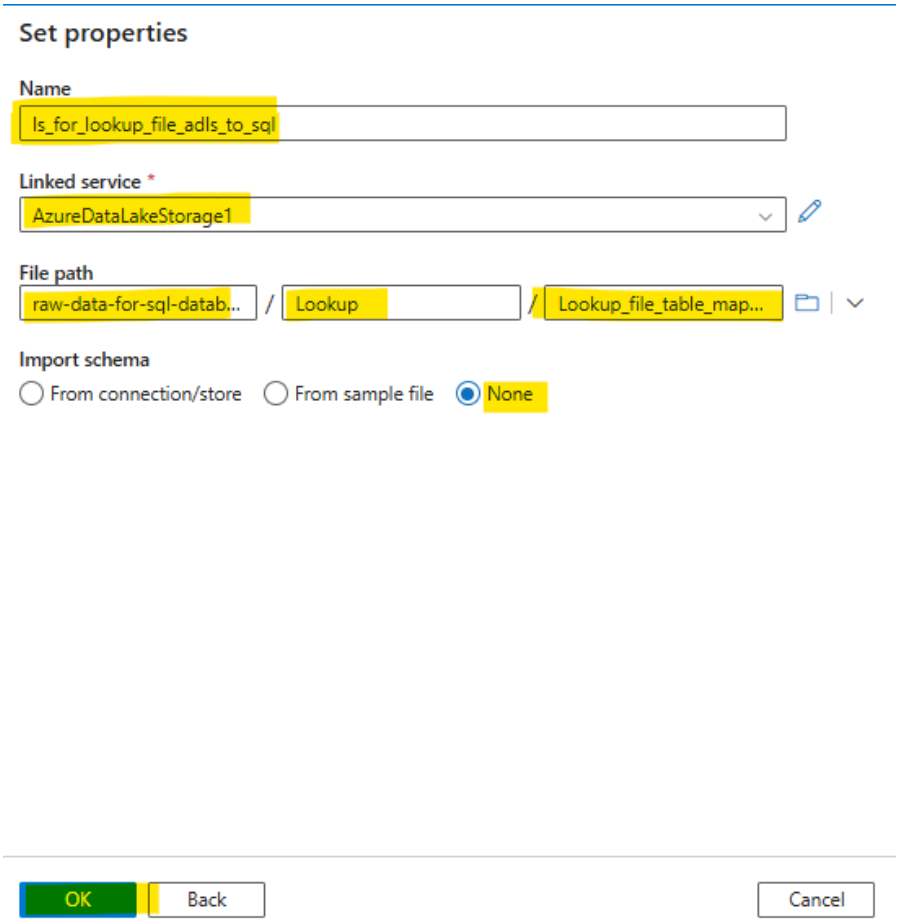
We will use the same generic dataset “generic_sql_ds” that we have created earlier.

=>For Lookup we will create a new dataset as shown below.

Select source as ADLS Gen2 storage, then in file format select json as our lookup file is a json file as shown below



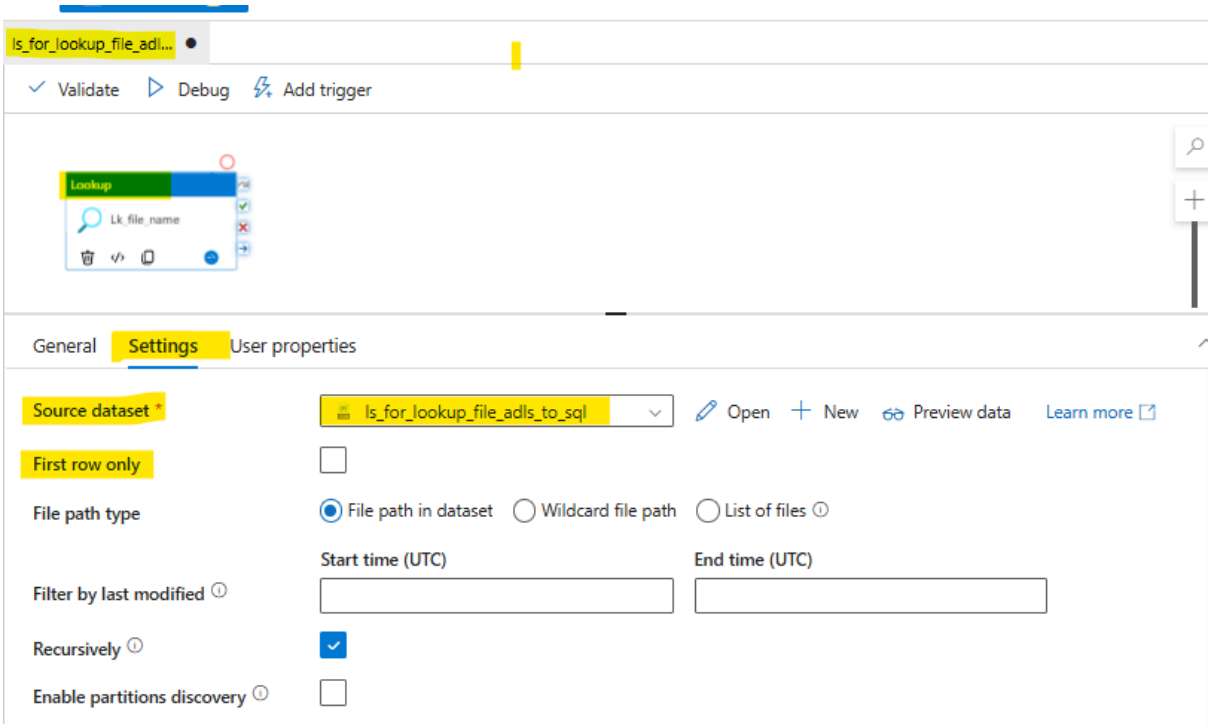




Steps to Configure the Pipeline:

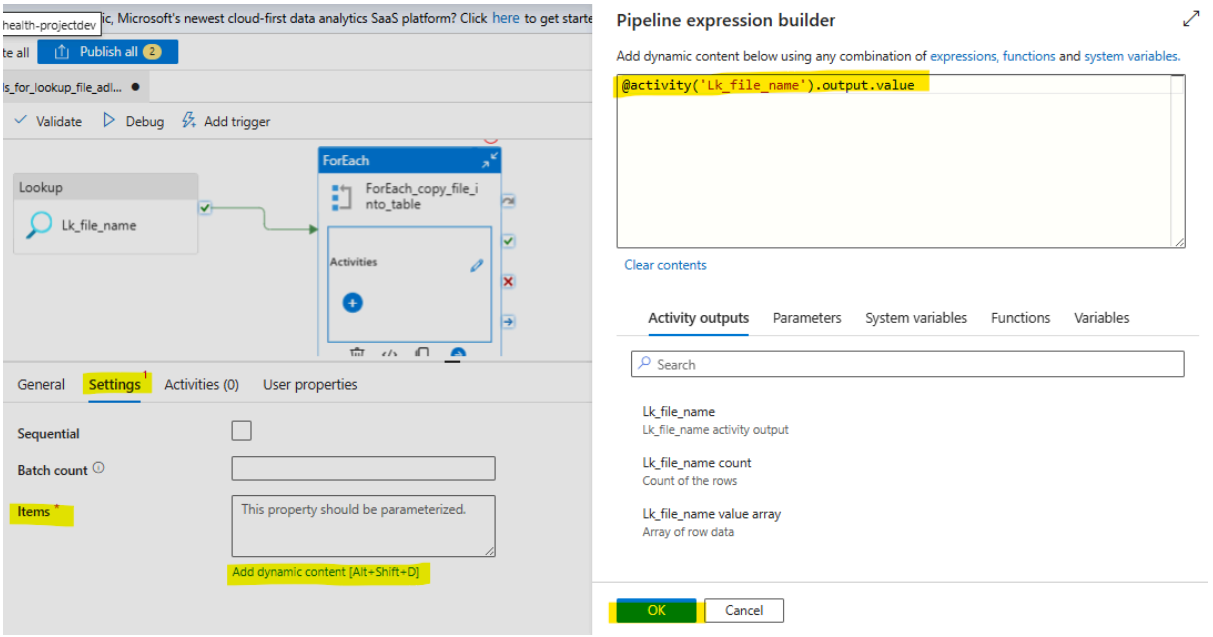
Add a Lookup Activity:

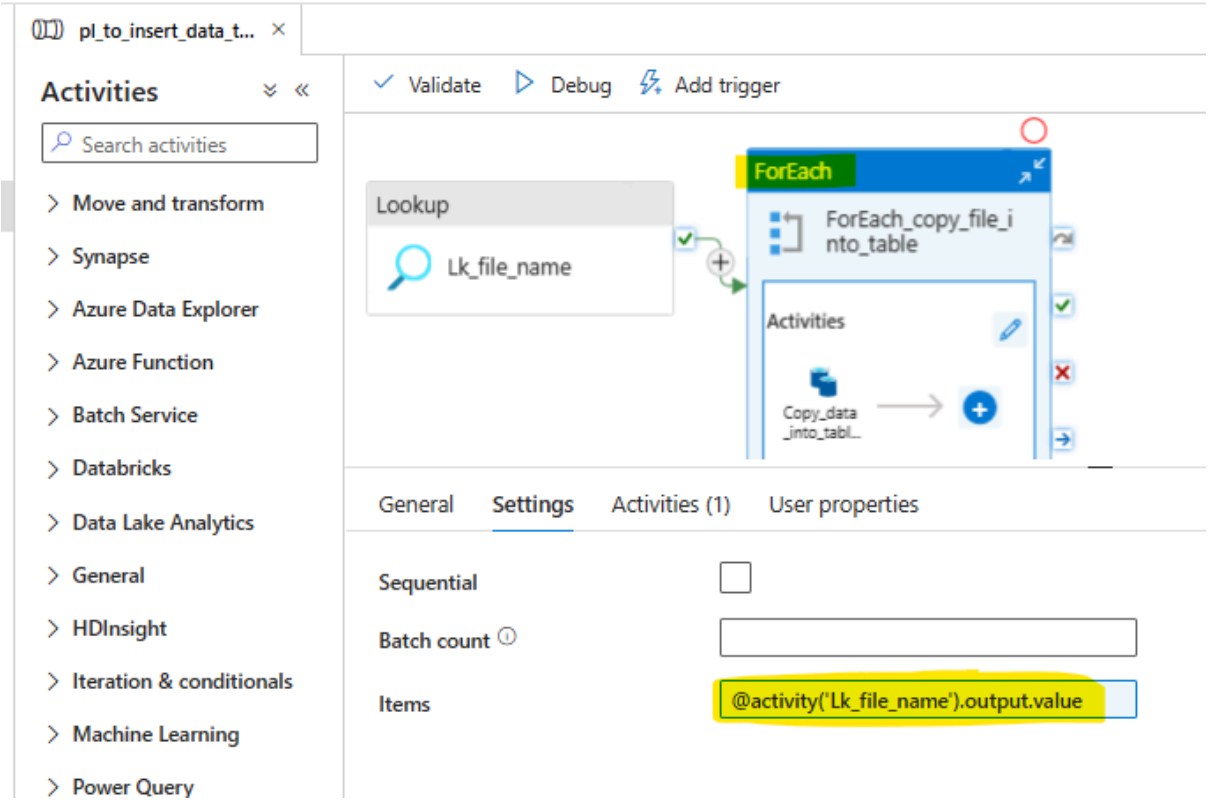
Drag a Lookup activity to the canvas. Point it to the mapping CSV dataset.Set First row only to false to read all rows. Refer below screenshot for more clarity.



Add a ForEach Activity:

Drag a ForEach activity and connect it to the Lookup activity.
Set its Items property to @activity('Lk_file_name').output.value
Refer below screenshot for more clarity.

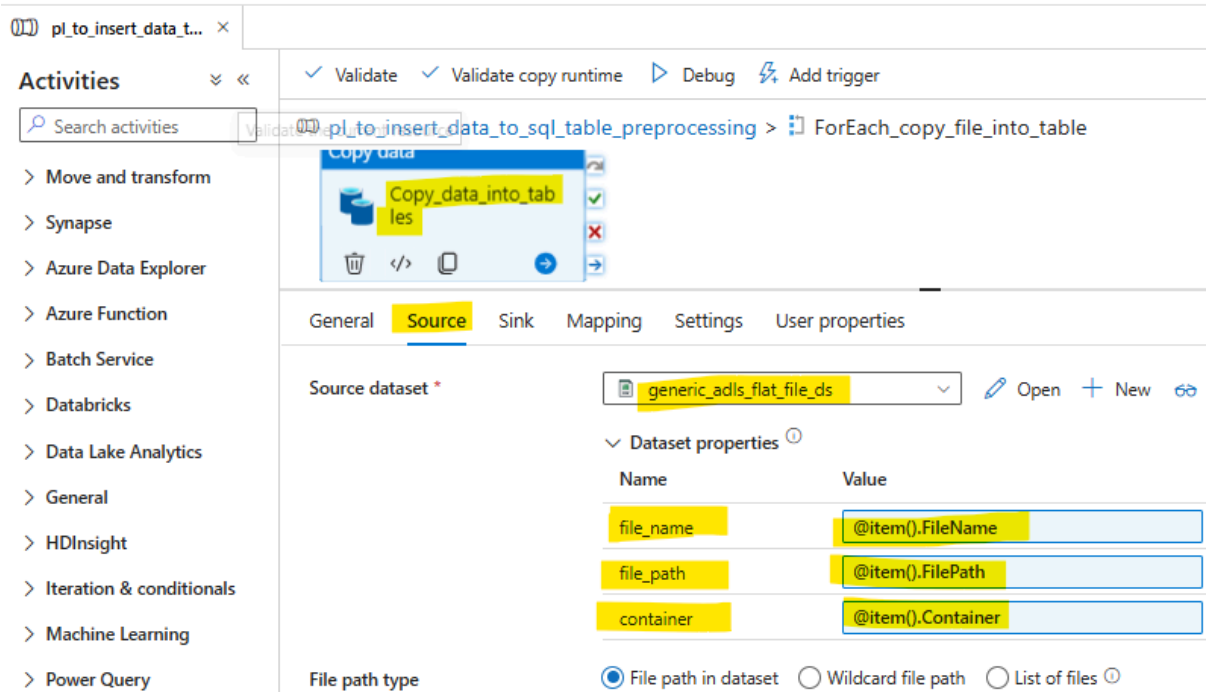




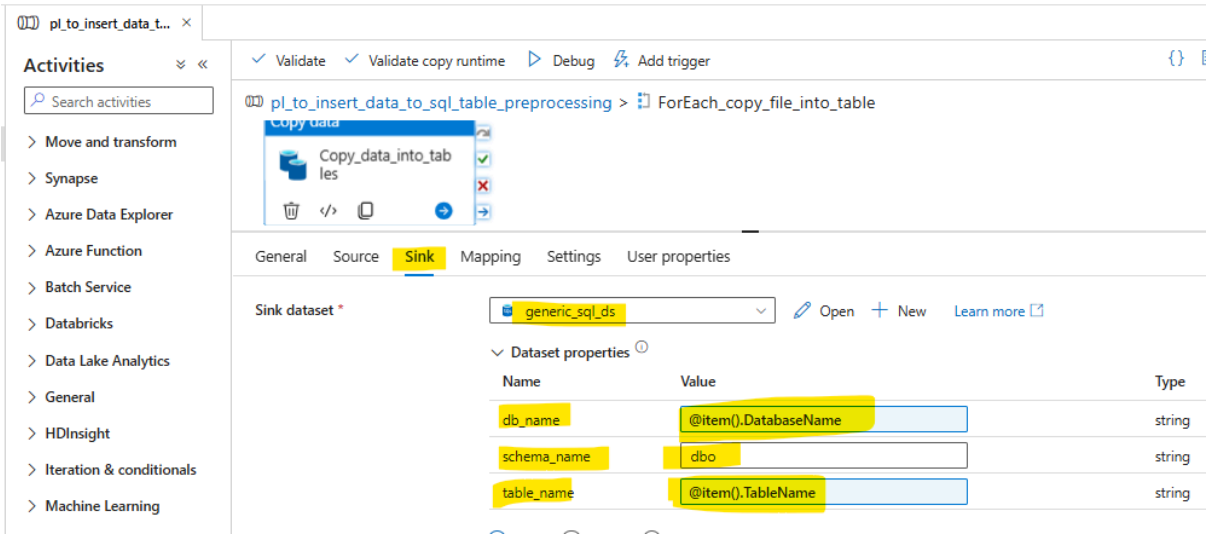
Configure the ForEach Activity:

Inside the ForEach activity, add a Copy Data activity.

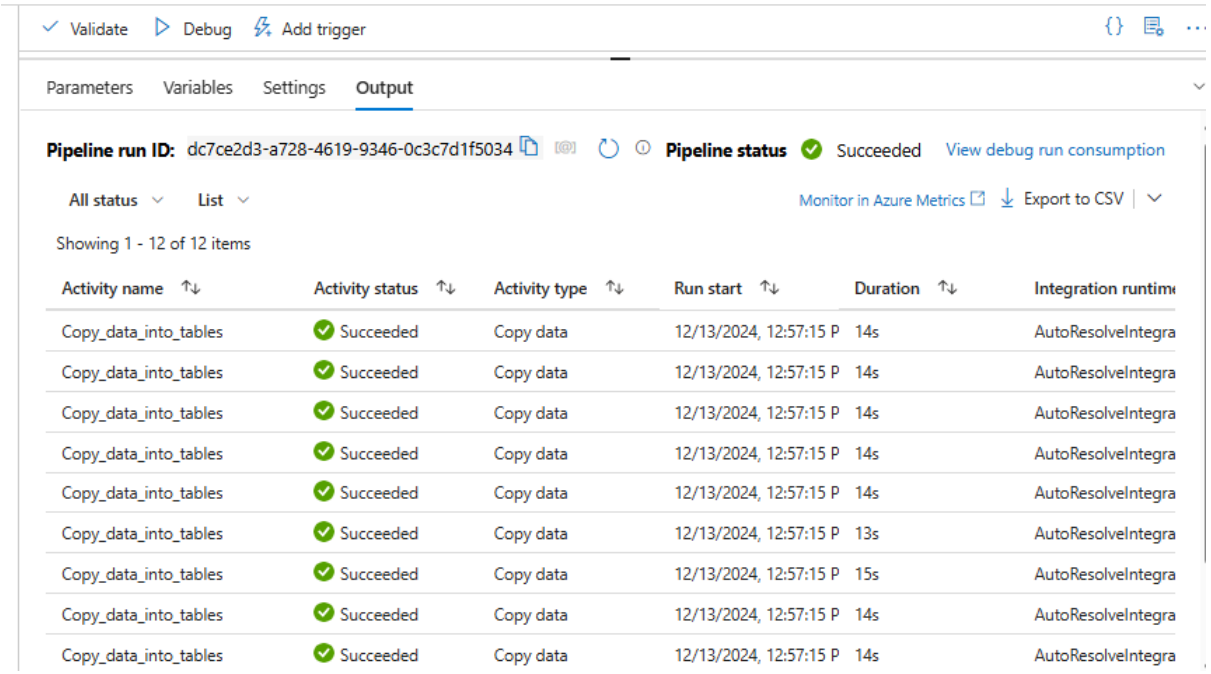
Source: Use the source dataset.



Sink: Use the destination dataset.



This pipeline will copy the data from the file into the tables in the sql database. On successfully running the pipeline we will get below output.

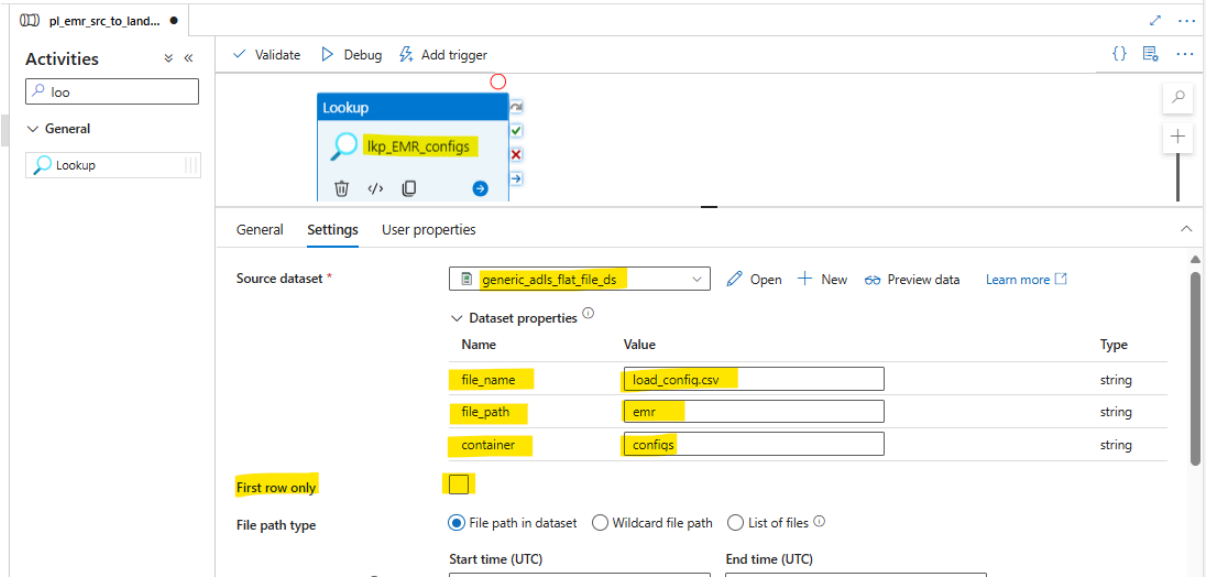


Activity name	Activity status	Activity type	Run start	Duration	Integration runtime
Copy_data_into_tables	Succeeded	Copy data	12/13/2024, 12:57:15 P	14s	AutoResolveIntegra
Copy_data_into_tables	Succeeded	Copy data	12/13/2024, 12:57:15 P	14s	AutoResolveIntegra
Copy_data_into_tables	Succeeded	Copy data	12/13/2024, 12:57:15 P	14s	AutoResolveIntegra
Copy_data_into_tables	Succeeded	Copy data	12/13/2024, 12:57:15 P	14s	AutoResolveIntegra
Copy_data_into_tables	Succeeded	Copy data	12/13/2024, 12:57:15 P	14s	AutoResolveIntegra
Copy_data_into_tables	Succeeded	Copy data	12/13/2024, 12:57:15 P	13s	AutoResolveIntegra
Copy_data_into_tables	Succeeded	Copy data	12/13/2024, 12:57:15 P	15s	AutoResolveIntegra
Copy_data_into_tables	Succeeded	Copy data	12/13/2024, 12:57:15 P	14s	AutoResolveIntegra
Copy_data_into_tables	Succeeded	Copy data	12/13/2024, 12:57:15 P	14s	AutoResolveIntegra

Pipeline to copy data from Azure Sql db to Landing Folder in ADLS Gen2

1. To read the config file we will use Lookup activity.

In this for source dataset will be for configs file and we will pass the parameter values as shown below.

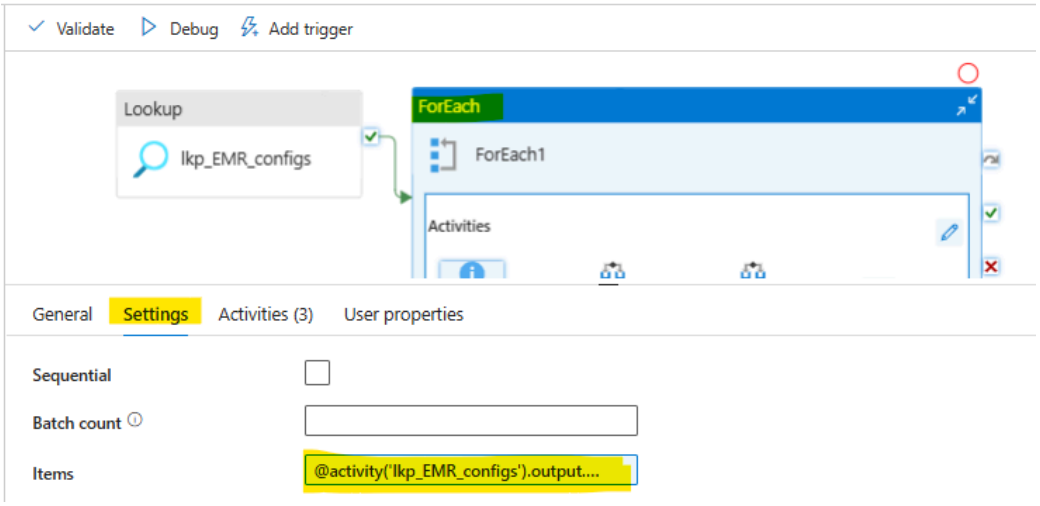


Then additionally we can preview the data.

2. In order to iterate through each row of configuration data we will use ForEach Activity.

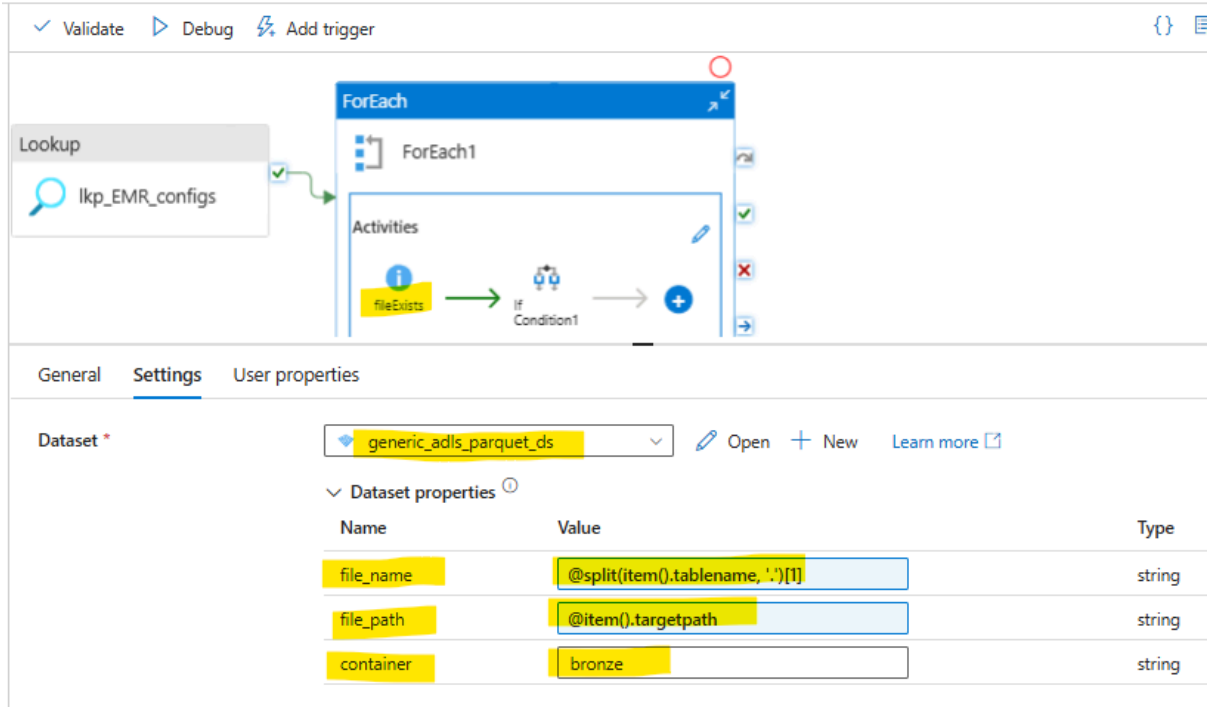
Processing Logic Within ForEach Activity:

@activity('lkp_EMR_configs').output.value



a. We will use get metadata activity in order to check whether file exists in Bronze container:

To file name we will use below logic - @split(item().tablename, '.')[1]
file_path is present in lookup file as targetpath
And container name we will explicitly mention as bronze as shown below

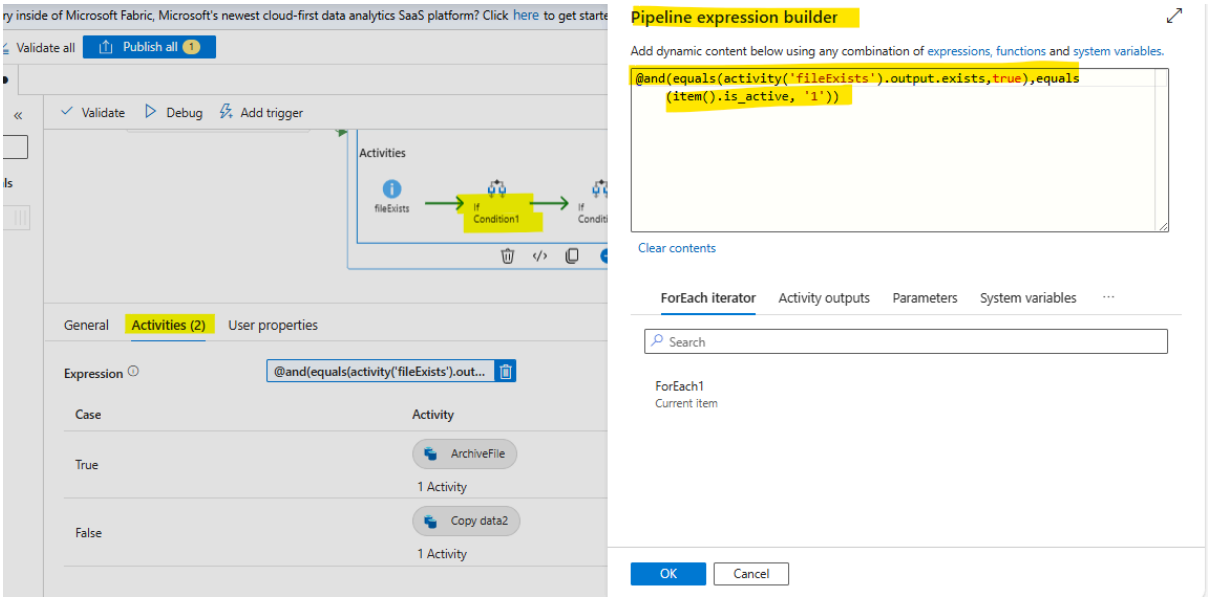


This will check if the file exists in the Bronze container. Based on the file's presence or absence, we will use an If Condition activity to determine the subsequent processing steps.

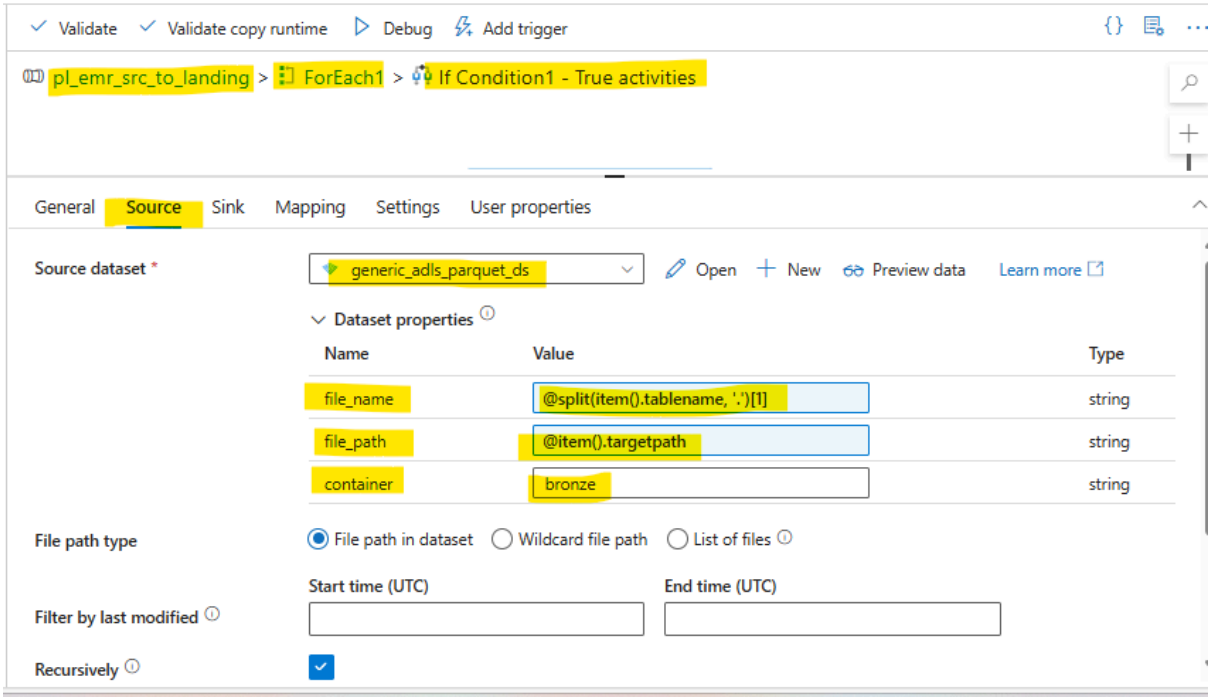
b. Use an If Condition activity based on the file's existence.

Condition 1: File Exists (True) => Move the file to the Archive folder.

condition:
@and(equals(activity('fileExists').output.exists,true),equals(item().is_active, '1'))



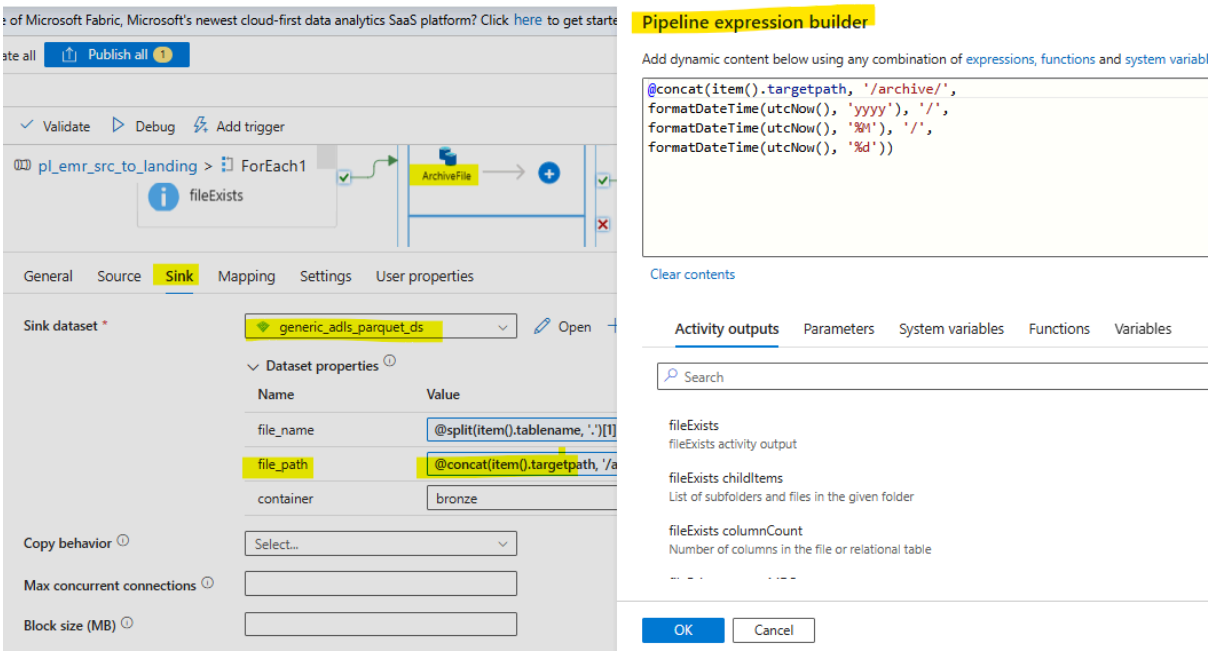
Source: Container: Bronze, Path: hosa, File: encounters



Target: Container: Bronze,

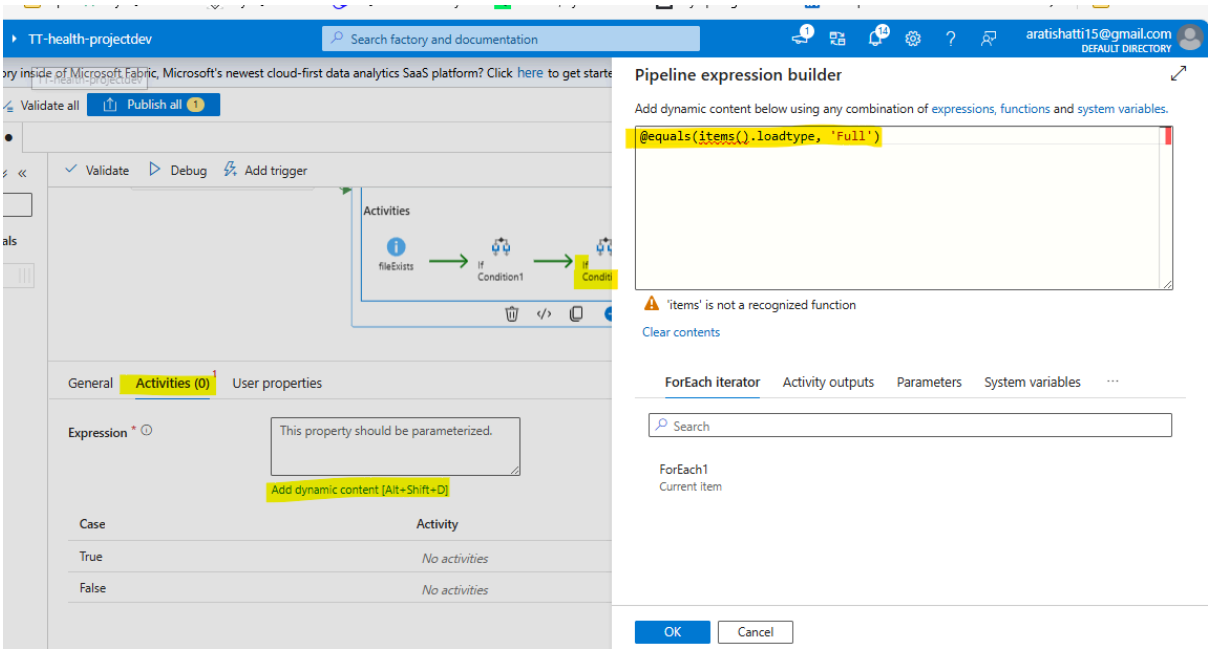
File_path -Path: hosa/archive/<year>/<month>/<day> =>
@concat(item().targetpath, '/archive/',
formatDateTime(utcNow(), 'yyyy'), '/',
formatDateTime(utcNow(), '%M'), '/',
formatDateTime(utcNow(), '%d'))

File_name - @split(item().tablename, '.')[1]



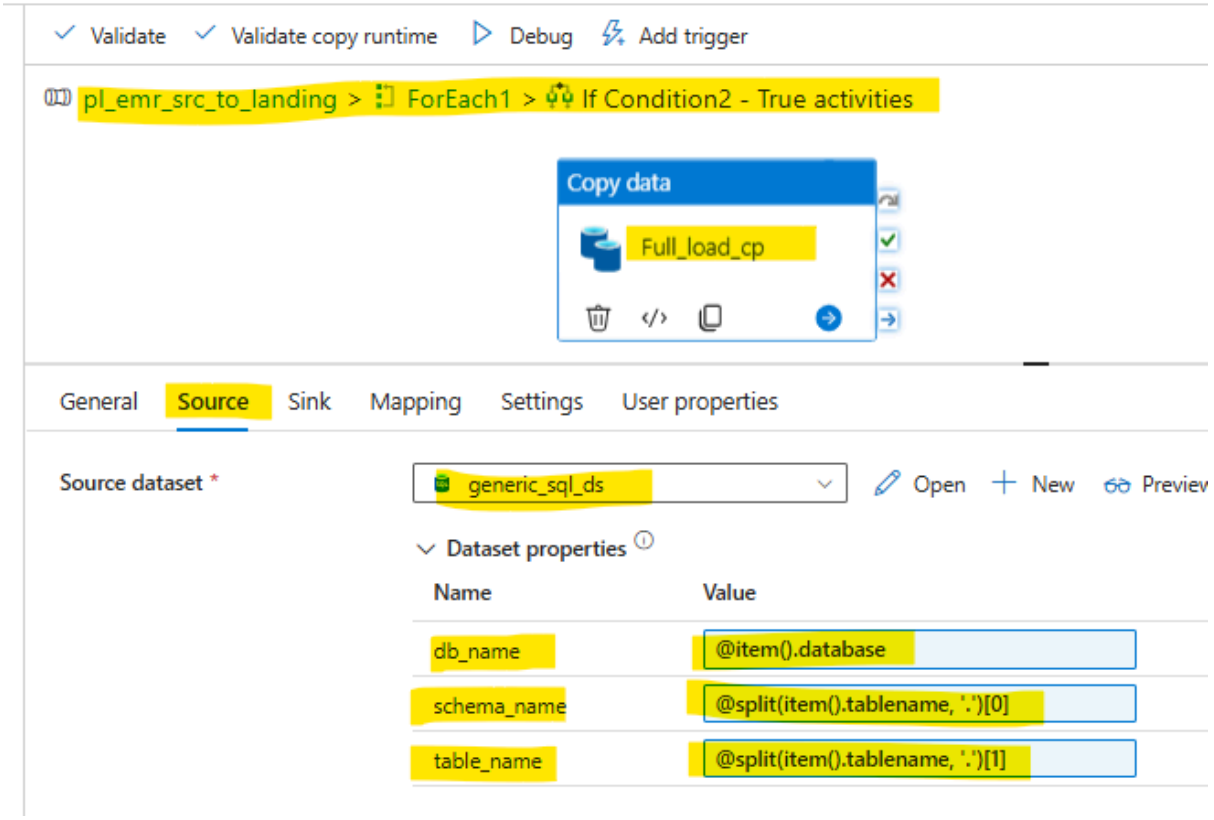
c. Determine if it's a full load or incremental load using If condition.

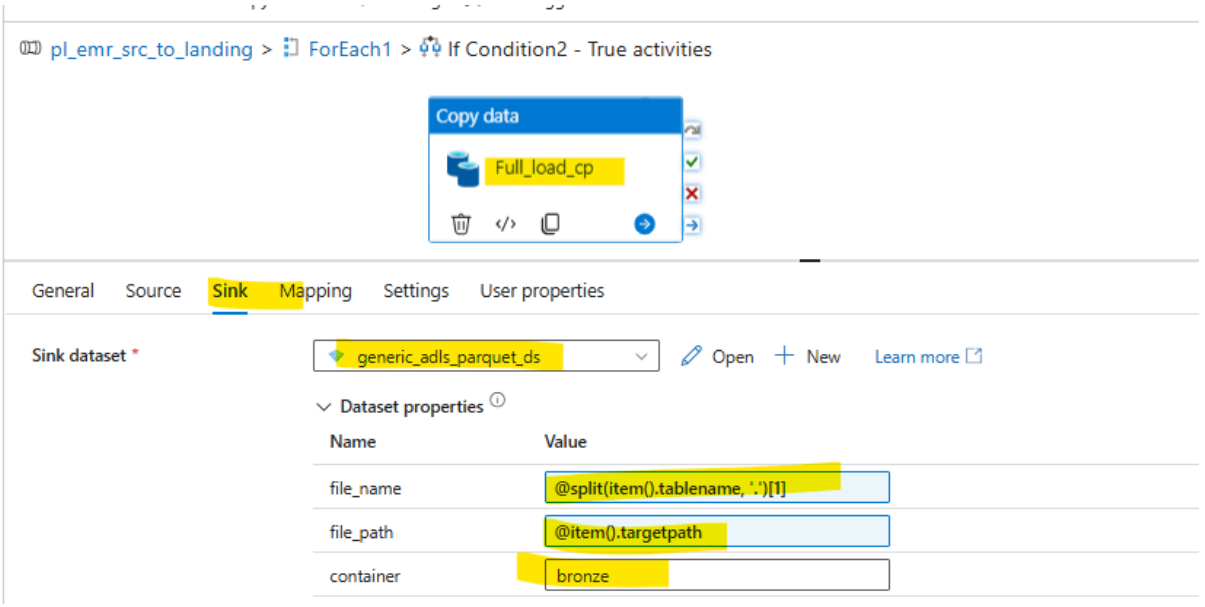
@equals(items().loadtype, 'Full')



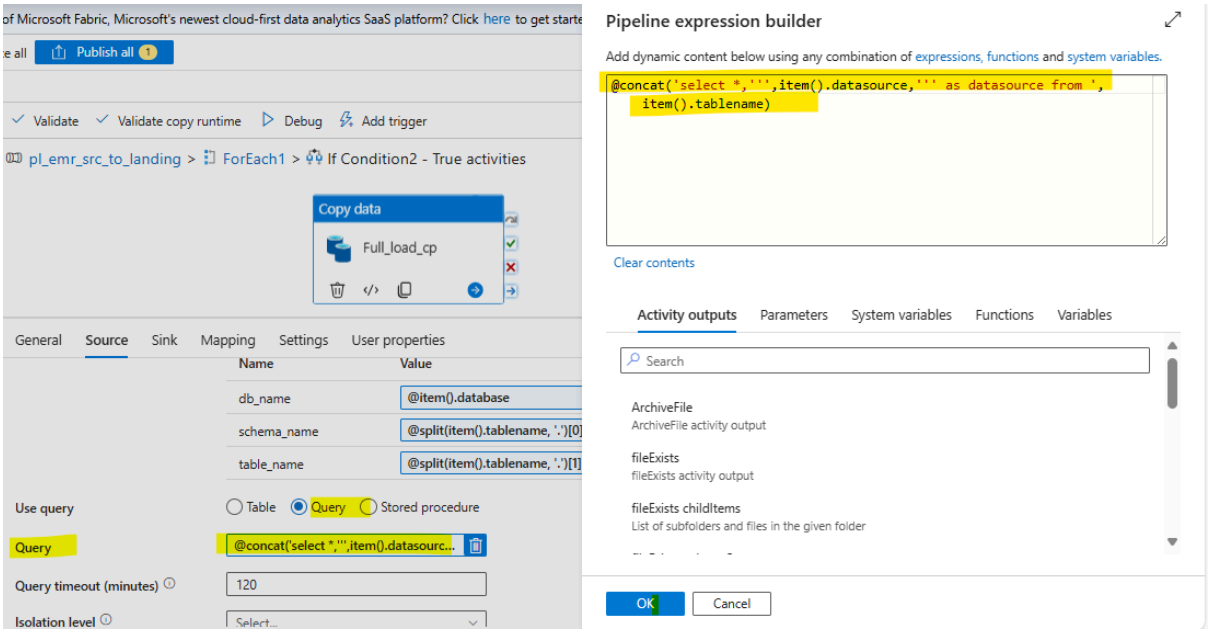
If "If condition" holds true => Full Load => Copy all data from the database table. => Enter Log details in the audit table:

Folder and File Structure
Bronze Container:
Source Path: bronze/hosa
Target Path for Data Loads: bronze/<target-path>



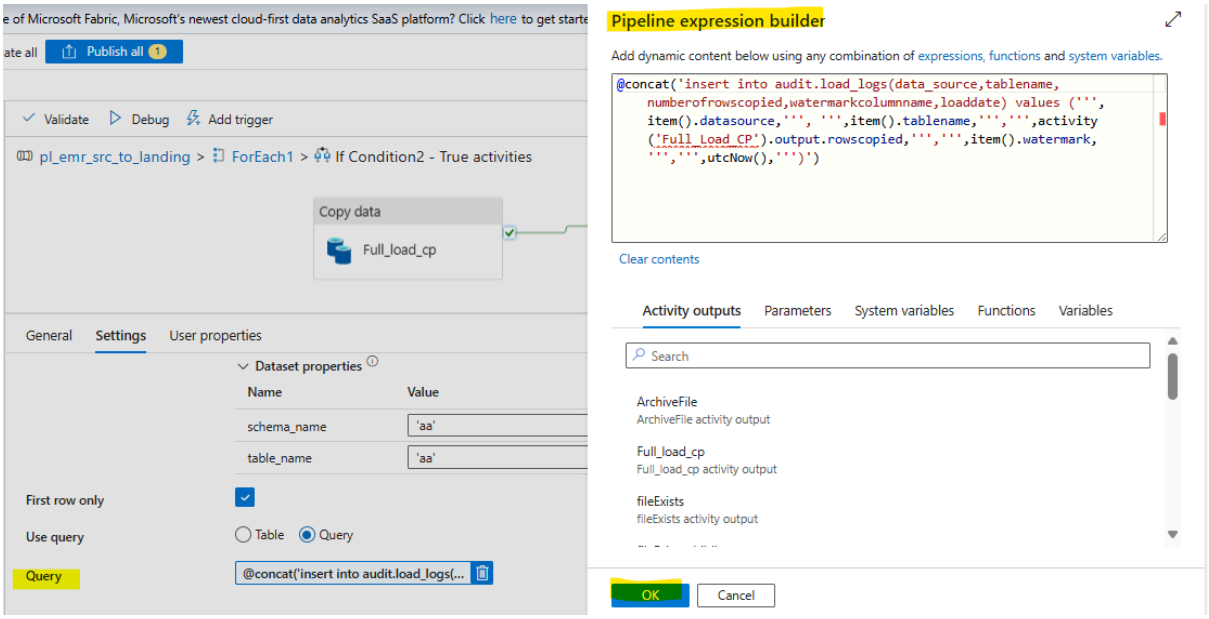


Query: @concat('select *,'",item().datasource,'" as datasource from ',item().tablename)



Enter Log details in the audit table:

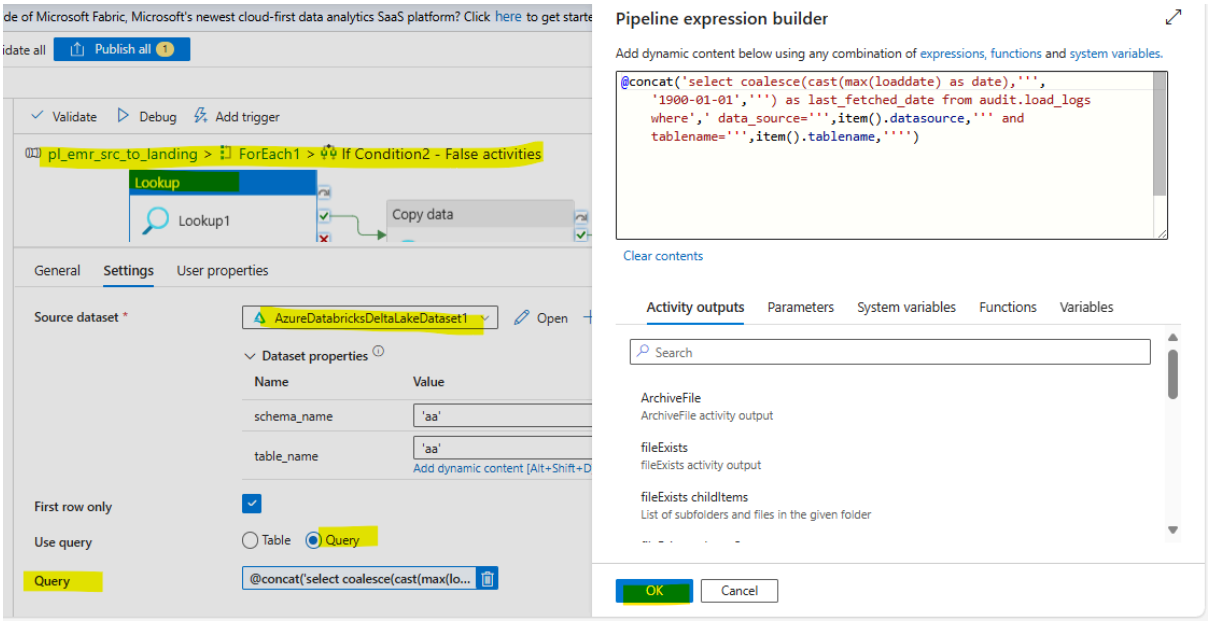
Query: @concat('insert into audit.load_logs(data_source,tablename,numberofrowscopied,watermarkcolumnname,loaddate) values (''',item().datasource,''',''',item().tablename,''','''',activity('Full_Load_CP').output.rowscopied,''','''',item().watermark,''','''',utcNow(),''')')



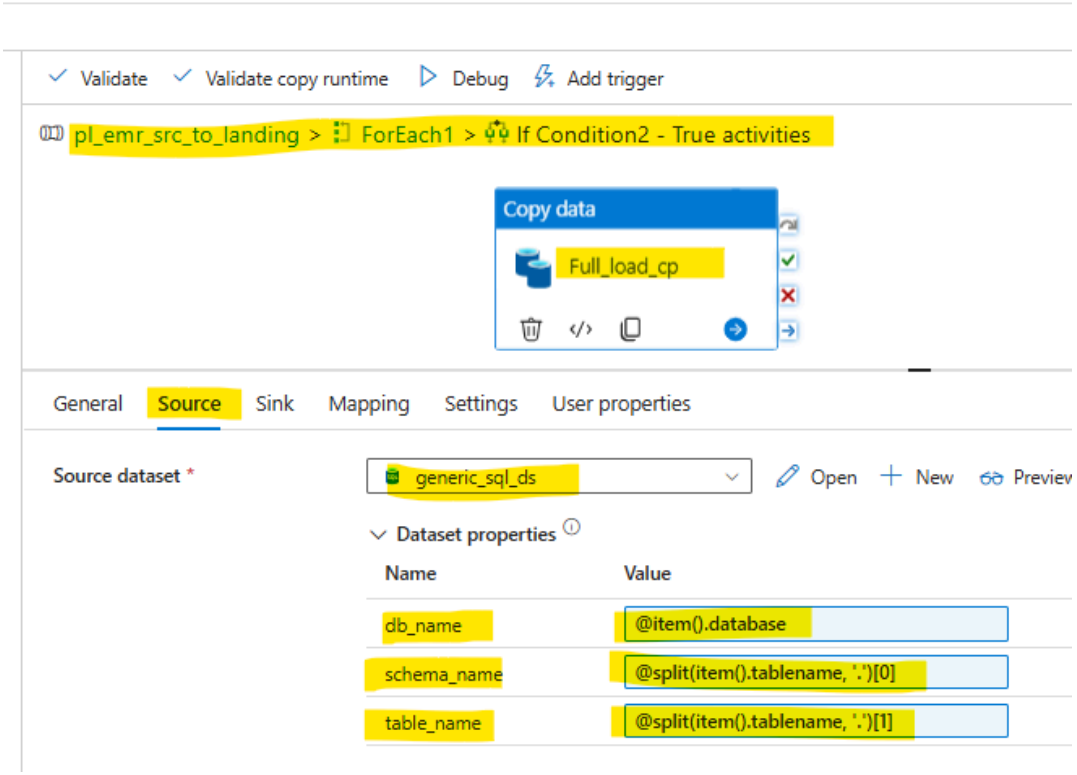
If condition is false => Incremental Load

(Fetch incremental data using the last fetched date) using Lookup=> Incremental load using copy activity =>Enter log details in the audit table:

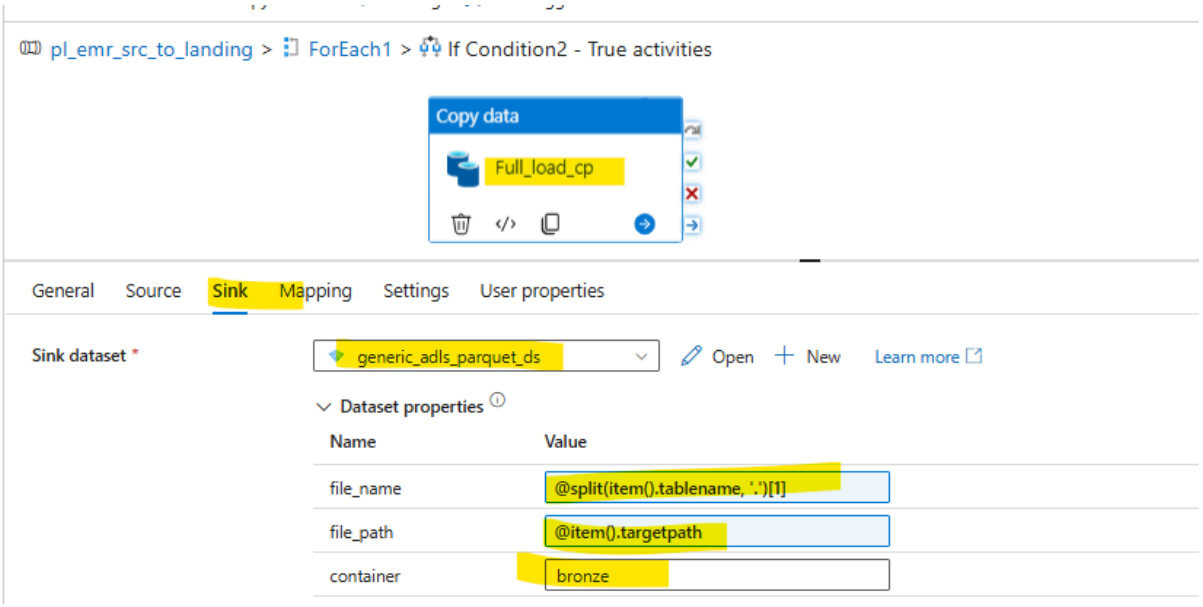
Lookup:



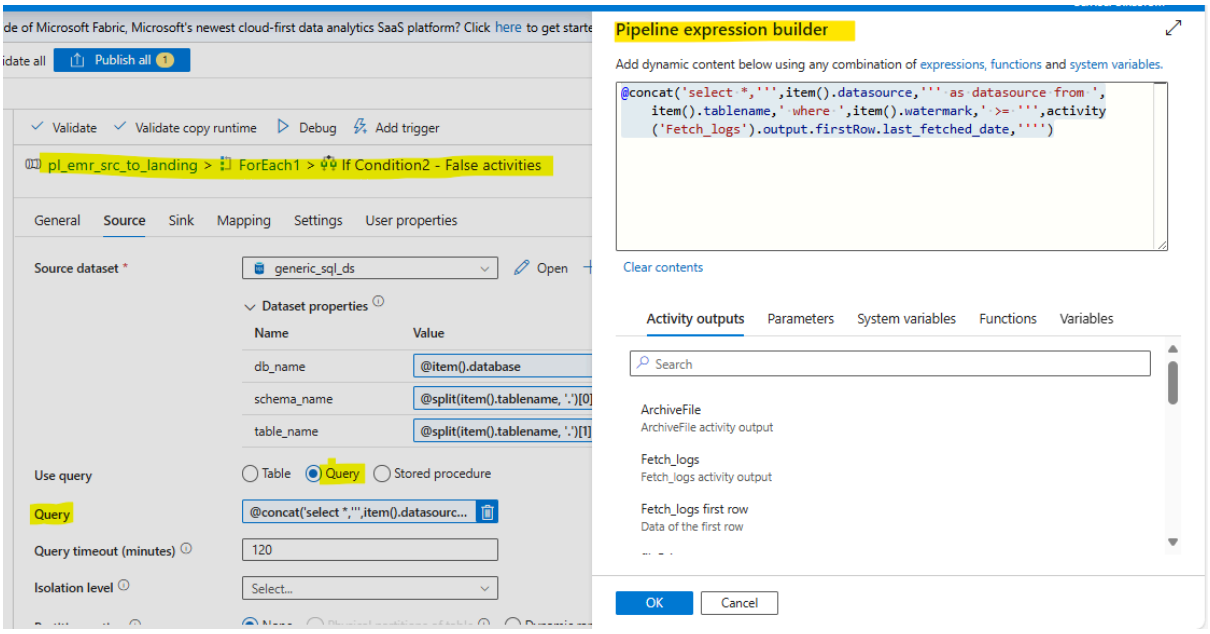
Incremental load:
Source Path: bronze/hosa



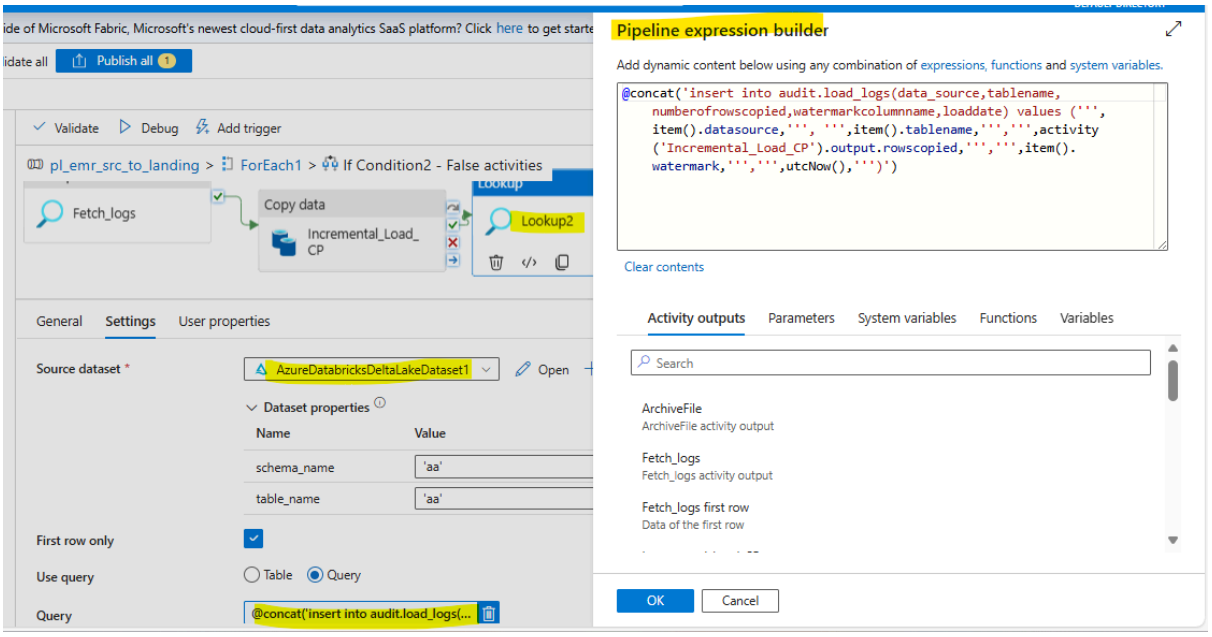
Target Path for Data Loads: bronze/<target-path>



Query: @concat('select *,'''',item().datasource,''' as datasource from ',item().tablename,' where ',item().watermark,' >= ''',activity('Fetch_logs').output.firstRow.last_fetched_date,'''')

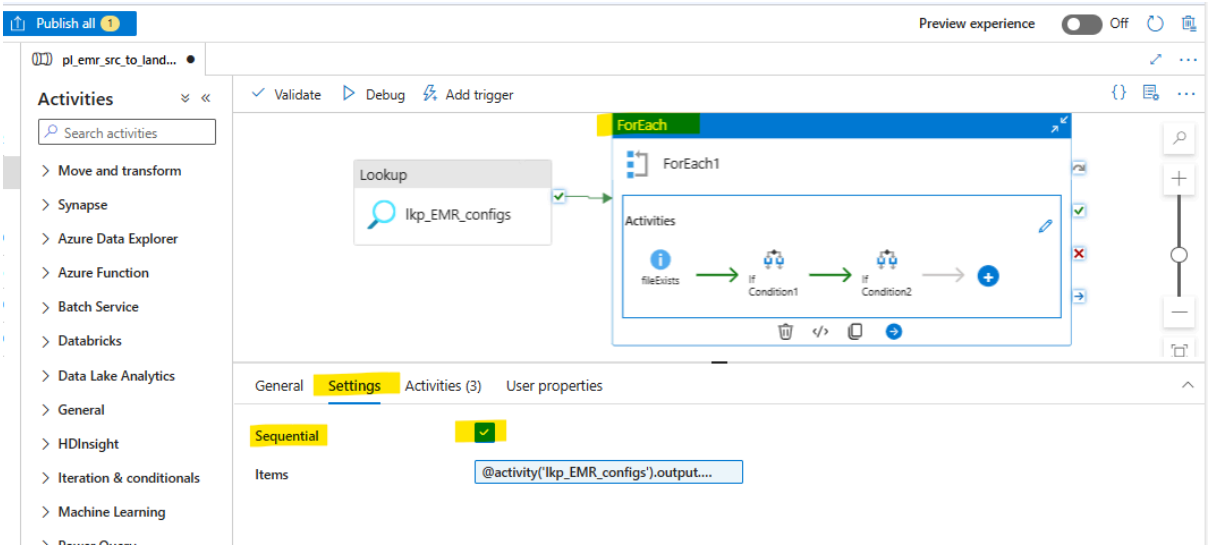


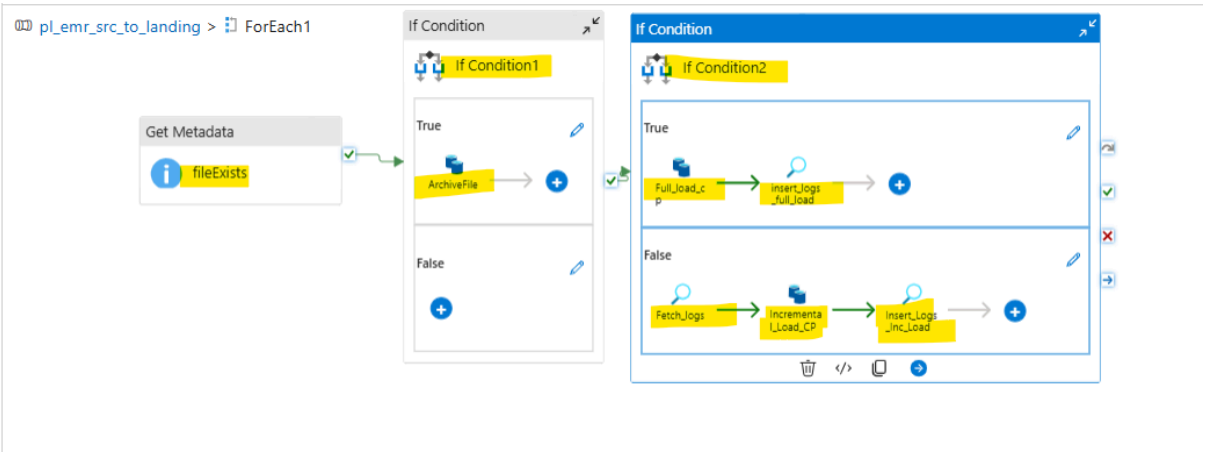
Lookup:



This is our complete pipeline:

Before running the pipeline for each activity select the “sequential” option as shown below.





But limitation with this pipeline is it is sequential which will we resolve in part 2

Part 2:

In this section, we will focus on improving our data pipeline and governance by implementing the following:

Clean and organize raw data into a structured format.
Apply Common Data Model (CDM) standards and implement Slowly Changing Dimensions (SCD2) for historical tracking.

Use Delta tables for efficient data storage and updates.

Build Fact and Dimension tables for better data analysis and reporting.

Secure sensitive data by integrating Azure Key Vault for managing secrets and credentials.

Standardize names across datasets, pipelines, and tables for better organization and understanding.

Optimize Azure Data Factory pipelines to run multiple processes at the same time, reducing execution time.

Integrate external APIs for dynamic data processing.
Handle Claims Data effectively, including implementing an is_active flag for tracking record statuses.

Transition from a local Hive Metastore to Databricks Unity Catalog for centralized metadata management and improved data governance.

We will first create new databricks workspace “tt-hc-adb-ws”, select the “Premium (+ Role-based access controls)” while creating workspace

Note: Also you can name the resource group as “TT-HeathProjectDev”

Also to organize the notebook we will create the folder as shown below.

Workspace › Users ›

trendytech.sumit501@outlook.com

Share | Create

Name	Type	Owner	Created at
Trendytech-Azure-Project featu	Git folder	Trendytech Insights	2024-12-02 22:02:28
1. Set up	Folder	Trendytech Insights	2024-11-25 00:14:09
2. API extracts	Folder	Trendytech Insights	2024-11-27 17:49:52
3. Silver	Folder	Trendytech Insights	2024-11-25 00:55:53
4. Gold	Folder	Trendytech Insights	2024-11-25 01:20:51
Gold queries	Query	Trendytech Insights	2024-12-03 00:27:54
keyvault test	Notebook	Trendytech Insights	2024-11-25 00:10:58

1. Set up:

Workspace › Users › trendytech.sumit501@outlook.com ›

1. Set up

Share | Create

Name	Type	Owner	Created at
1. audit_ddl	Notebook	Trendytech Insights	2024-11-25 00:14:14
2. adls_mount	Notebook	Trendytech Insights	2024-11-25 00:16:49

2. API extracts

Workspace › Users › trendytech.sumit501@outlook.com ›

2. API extracts

Share | Create

Name	Type	Owner	Created at
ICD Code API extract	Notebook	Trendytech Insights	2024-11-28 08:57:55
NPI API extract	Notebook	Trendytech Insights	2024-11-27 17:50:07

3. Silver

Workspace › Users › trendytech.sumit501@outlook.com ›

3. Silver ☆

Share | Create

Name	Type	Owner	Created at
Claims	Notebook	Trendytech Insights	2024-11-25 00:55:54
CPT codes	Notebook	Trendytech Insights	2024-11-28 23:13:09
Departments_F	Notebook	Trendytech Insights	2024-11-25 00:55:55
Encounters	Notebook	Trendytech Insights	2024-11-25 00:55:55
ICD Code	Notebook	Trendytech Insights	2024-11-28 09:08:35
NPI	Notebook	Trendytech Insights	2024-11-26 14:42:54
Patient	Notebook	Trendytech Insights	2024-11-25 00:55:54
Providers_F	Notebook	Trendytech Insights	2024-11-25 00:55:55
Transactions	Notebook	Trendytech Insights	2024-11-25 00:55:53

4. Gold

Workspace › Users › trendytech.sumit501@outlook.com ›

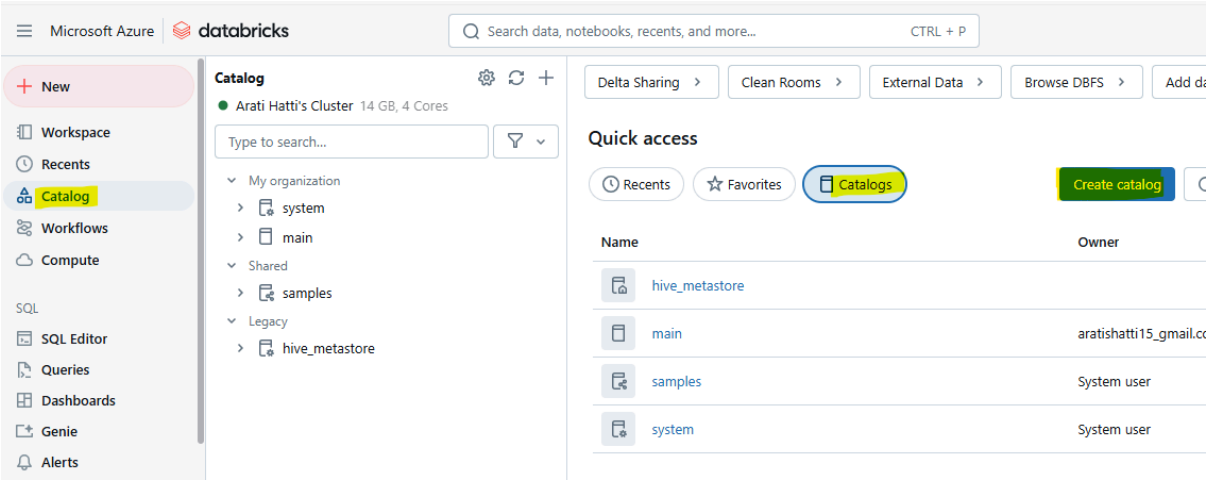
4. Gold ☆

Share | Create

Name	Type	Owner	Created at
dim_cpt_code	Notebook	Trendytech Insights	2024-11-28 23:36:42
dim_department	Notebook	Trendytech Insights	2024-11-25 01:42:29
dim_icd_code	Notebook	Trendytech Insights	2024-11-28 09:11:38
dim_npi	Notebook	Trendytech Insights	2024-11-28 09:15:54
dim_patient	Notebook	Trendytech Insights	2024-11-25 01:20:59
dim_provider	Notebook	Trendytech Insights	2024-11-25 01:29:07
fact_transaction	Notebook	Trendytech Insights	2024-11-26 14:22:04

Note: After creating the databricks workspace enable the DBFS.

Create the catalog “tt-hc-adb-ws” as shown below.



Now we will create the audit database using as shown below